Universitatea
Transilvania
din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

PROIECT DE DIPLOMĂ

Conducător științific:COCIAȘ Tiberiu Teodor
Titulatura. Conf.dr.inginer

Absolvent:
CONDREA Laurențiu Daniel

BRAȘOV, 2025

Departamentul Automatică și tehnologia informației
Programul de studii: Robotică

CONDREA Laurențiu Daniel

Proiectarea unui braț robotic pentru sortarea deșeurilor reciclabile

Conducător științific:
Conf.dr.inginer COCIAȘ Tiberiu Teodor

Brașov, 2025

Cuprins

Listă de acronime 6
Lista de figuri, tabele și coduri sursă 7
1 Introducere 9
1.1 Contextul lucrării 9
1.2 Stagiul actual in literatura de specialitate 10
1.3 Metodologia de lucru 11
1.4 Motivație 12
1.5 Structura lucrării 12
2 Noțiuni fundamentale 14
2.1 Yolov8 14
2.1.1 Backbone-ul rețelei 16
2.1.2 Neck-ul rețelei 17
2.1.3 Head-ul rețelei 17
2.2 Aruco Markers 18
2.2.1 Estimarea poziției și orientării 18
2.2.2 Procesul de detectare 19
2.3 Metodele modelului geometric și cinematic 19
2.4 Servomotoare 21
2.4.1 Principiul de funcționare al unui servomotor 22
2.4.2 Tipuri de servo-motoare 22
2.5 Parametrii de calibrare a camerei 23
3 IMPLEMENTARE 24
3.1 Descrierea problemei 24
3.2 Modelarea Sistemului 25
3.3 Antrenarea si evaluarea modelului Yolov8 27
3.4 Calibrarea Camerei 31
3.5 Modelul geometric si cinematic al manipulatorului 34
3.5.1 Geometria directă 36
3.5.2 Reprezentarea configurației 39
3.5.3 Pseudo-animație 40
3.5.4 Calculul Jacobianului 42

3.5.5 Metoda de geometria inversă 45
3.5.6 Pick and Place 48
3.5.7 Cinematică directă 50
3.5.8 Cinematică inversă 50
3.5.9 Volumul de lucru 51
3.6 Componente folosite 52
4 Rezultate Experimentale 56
4.1 Testarea detectiei cu YOLOv8n 56
4.1.1 Cadrul experimental 56
4.1.2 Evaluarea rezultatelor 57
4.2 Testarea markerilor ArUco pentru calibrare și poziționare 57
4.2.1 Cadrul experimental 57
5 Concluzii 59
5.1 Concluzii generale 59
5.2 Contribuții personale 60
5.3 Posibilitati de dezvoltare ulterioara 60
6 Bibliografie 62
Rezumat 64
Abstract 65
DECLARAȚIE PRIVIND ORIGINALITATEA 66

LISTĂ DE ACRONIME

CAD – Computer-Aided Design

FRT – Formalismul Reperului Translatat

LSTM – Long Short-Term Memory

mAp – mean Average Precision

PWM – Pulse Width Modulation

RL – Reinforcement Learning

SPPF – Spatial Pyramid Pooling Fast

YOLO – You Only Look Once

LISTA DE FIGURI, TABELE ȘI CODURI SURSĂ

FIGURI

Figura 2.1.1 Arhitectura rețelei de vedere artificiala YOLOv

Figura 2.1.2 Backbone-ul rețelei

Figura 2.1.3 Neck-ul şi Head-ul rețelei

Figura 2.2.1 Markere ArUco

Figura 2.4.1 Construcția unui servomotor

Figura 3.2.1 Diagrama sistemului

Figura 3.3.1 Etichetarea manuală a imaginilor

Figura 3.3.2 Setul de date

Figura 3.3.3 Rezultate 20 de epoci

Figura 3.3.4 Rezultate 100 de epoci

Figura 3.4.1 Cameră Canyon C3

Figura 3.4.2 Tablă de șah

Figura 3.4.3 Colțuri detectate

Figura 3.4.4 Translația si rotația camerei

Figura 3.5.1 Desen 5DOF

Figura 3.5.2 Baza-Cupla1

Figura 3.5.3 Cupla1-Cupla2

Figura 3.5.4 Cupla2-Cupla3

Figura 3.5.5 Cupla3-Cupla4

Figura 3.5.6 Cupla4-Cupla5

Figura 3.5.7 Cupla5-Efector

Figura 3.5.8 Reprezentarea configurației (toate cuplele se află în poziția 0)

3.5.9 Animație (robotul pleacă de la poziția 0 in toate cuplele și trebuie să se aplice pi/2 pe cuplele q_3 $și$ q_4)

Figura 3.5.10 Animație (q_3 a ajuns la pi/2, iar acum se modifică q_4)

Figura 3.5.11 Animație (q_4 a ajuns la pi/2 și robotul s-a mișcat conform impunerii)

Figura 3.5.12 Geometrie inversă (robotul pleacă de la toate cuplele fiind la pi/4 și trebuie să ajungă în P_e, adică punctul roșu)
Figura 3.5.13 Geometrie inversă (mișcările robotului la jumătate din iterații)
Figura 3.5.14 Geometrie inversă (robotul a ajuns în P_e)
Figura 3.5.15 Poziția Pick and Place
Figura 3.5.16 Viteza Pick and Place
Figura 3.5.17 Accelerația Pick and Place
Figura 3.5.18 Cinematica direct
Figura 3.5.19 Viteza generalizată din cinematică inversă
Figura 3.5.20 Viteza din traiectoria de tip Pick and Place
Figura 3.5.21 Volumul de lucru (vedere de sus)
Figura 3.6.1 Servomotor MG966R
Figura 3.6.2 Raspberry Pi5
Figura 3.6.3 Sursă 5V 20A
Figura 3.6.4 Vedere de ansamblu asupra componentelor sistemului
Figura 4.1.1 Detecție PET
Figura 4.2.2 Cameră calibrată
Figura 4.2.2 Cameră necalibrată

TABELE

Tabelul 2.1.1 Tipuri de YOLOv8
Tabelul 2.1.2 Sarcini si moduri suportate
Tabelul 2.3.1 Metodele modelului geometric și cinematic
Tabel 3.4.1 Caracteristici cameră web
Tabel 3.6.1 Caracteristici servo-motor MG966R
Tabel 3.6.2 Caracteristici motor DC și driver L9110

1 INTRODUCERE

1.1 Contextul lucrării

Suntem într-o perioadă de dezvoltare tehnologică accelerată, in care soluțiile automatizate devin din ce în ce mai răspândite in domenii variate, inclusiv in gestionarea deșeurilor si protejarea mediului. În acest context, integrarea roboților în procese precum sortarea si reciclarea deșeurilor reprezintă o soluție eficienta.

Roboții sunt tot mai prezenți in diverse domenii de activitate, datorită versatilității si eficienței lor. Aceștia pot fi împărțiți in funcție de mobilitate, în roboți imobili, fixați pe o structură, și roboți mobili, capabili să se deplaseze. Roboții imobili sunt utilizați in principal pentru sarcini de asamblare, modelare sau prelucrare a obiectelor și necesită un grad ridicat de precizie în manipulare, pe de altă parte, roboții mobili sunt folosiți pentru transportul si manipularea obiectelor in spații variate, având ca cerința principală o precizie ridicata in localizare si navigație.

Proiectul de față își propune realizarea unui robot mobil, echipat cu un braț manipulator, capabil să identifice, să sorteze și să plaseze corect deșeurile în funcție de materialul din care sunt realizate: carton, aluminiu, plastic și sticlă. Identificarea tipului de material este realizată cu ajutorul unei camere video conectate la un algoritm de clasificare bazat pe învățare automată. Aceasta a fost antrenată să recunoască vizual principalele tipuri de deșeuri reciclabile, pe baza unor seturi de date relevante. Pentru localizarea obiectelor și poziționarea precisă a brațului manipulator, sunt utilizate markerele ArUco, care identifică rapid poziția și orientarea deșeurilor în spațiu. Generarea traiectoriei brațului robotic se bazează pe utilizarea matricei Jacobiene, care permite calculul mișcărilor articulațiilor în funcție de poziția dorită a efectorului final. Prin acest proiect, se urmărește nu doar realizarea unui sistem funcțional, dar și demonstrarea aplicabilității inteligenței artificiale și a roboticii în domeniul reciclării.

1.2 Stagiul actual in literatura de specialitate

Automatizarea procesului de sortare a deșeurilor a devenit o direcție importantă de cercetare, cu accent pe dezvoltarea unor sisteme robotizate inteligente capabile să identifice, localizeze și manipuleze diverse tipuri de materiale reciclabile. În literatura de specialitate se regăsesc numeroase soluții, atât fixe cât și mobile, care integrează vederea artificială, inteligență artificială și sisteme de manipulare pentru a atinge acest scop.

Un exemplu bine-cunoscut este ZenRobotics, un sistem industrial care utilizează brațe robotice staționare și camere multispectrale pentru a identifica și sorta automat materialele de pe o bandă transportoare. Sistemul recunoaște materiale precum metal, plastic, lemn sau beton folosind algoritmi de clasificare pe baza imaginilor și a datelor senzoriale (detecție de metale, spectrometrie în infraroșu), calculând poziția obiectelor prin detecție 2D corelată cu mișcarea benzii, estimând astfel poziția exactă și momentul optim pentru ca brațul robotic să preia obiectul. [1]

Waste Robotics WR-1 se diferențiază prin mobilitatea autonomă a platformei, care îi permite să navigheze liber în spațiul de lucru, extinzând aria de operare dincolo de o bandă transportoare fixă. Această mobilitate, combinată cu un braț robotic cu multiple grade de libertate și senzori 3D avansați, oferă o flexibilitate crescută în manipularea obiectelor în medii variabile și mai puțin structurate. În plus, integrarea strânsă între navigație și manipulare necesită algoritmi avansați de coordonare, crescând eficiența în identificarea, localizarea și manipularea materialelor reciclabile în spațiul tridimensional. [2]

Recycleye se diferențiază prin integrarea unei platforme software cloud avansate, care permite monitorizarea în timp real, analiza datelor și optimizarea continuă a procesului de sortare. Recycleye oferă un sistem complet de management al procesului de reciclare, facilitând ajustarea automată a criteriilor de clasificare și transparența operațională. De asemenea, utilizează tehnologii avansate de segmentare și recunoaștere a obiectelor, combinate cu senzori de înaltă rezoluție, pentru o detecție extrem de precisă și adaptabilă la diverse tipuri de materiale reciclabile. [3]

Google Robotics și Everyday Robots au dezvoltat un sistem de sortare automată a deșeurilor în clădiri de birouri, folosind o flotă de roboți mobili cu brațe robotice și camere RGB. Sistemul se bazează pe deep reinforcement learning, care învață din simulări și experiență reală, pentru a sorta corect obiectele. Arhitectura include o rețea cu memorie LSTM și viziune semantică (ShapeMask). Testat pe 23 de roboți timp de doi ani, sistemul a redus greșelile de

sortare cu peste 84% și contaminarea deșeurilor cu 53%, demonstrând eficiența deep RL la scară reală. [4]

Un alt exemplu este ConvoWaste, o mașină de sortare automată a deșeurilor dezvoltată de Universitatea AIUB, Bangladesh. Sistemul folosește un model CNN bazat pe Inception-ResNet V2 pentru clasificarea a 6 tipuri de deșeuri (plastic, metal, sticlă, medical, organic, e-waste) pe o bandă transportoare. După clasificare, un servomotor direcționează deșeurile către coșul corespunzător. Mașina integrează senzori ultrasonici pentru volum și comunicare GSM pentru notificări, fiind controlată și printr-o aplicație Android via Bluetooth. Modelul a atins o precizie de 98,3% și oferă o soluție accesibilă și scalabilă. [5]

1.3 Metodologia de lucru

Obiectivul principal al acestei lucrări este dezvoltarea unui robot mobil echipat cu un braț robotic, capabil să identifice, să clasifice și să sorteze deșeuri în funcție de materialul acestora (plastic, sticlă, carton, aluminiu). Sistemul integrează procesare de imagine, localizare în timp real și planificare de traiectorii, pentru a realiza sarcini complexe într-un mediu controlat.

Pentru atingerea acestui obiectiv general, metodologia de lucru a fost structurată în următoarele etape și obiective specifice:

1. Reprezentarea mediului înconjurător

- Mediul de lucru al robotului este capturat în timp real prin intermediul unei camere video montate pe platforma mobilă.
- Informațiile vizuale sunt utilizate atât pentru recunoașterea obiectelor, cât și pentru identificarea markerelor ArUco necesare localizării.

2. Localizarea și orientarea robotului în scenă

- Markerele ArUco sunt folosite pentru a determina poziția și orientarea robotului mobil și a obiectelor din scenă.
- Folosind biblioteca OpenCV, poziția markerelor este calculată prin estimarea pose-ului, iar coordonatele sunt raportate la un sistem global de referință.

3. Clasificarea deșeurilor în funcție de material

- Recunoașterea și clasificarea obiectelor se realizează cu ajutorul unei camere și a unui model de tip YOLO antrenat pentru a distinge între diferite tipuri de materiale: plastic, sticlă, carton, aluminiu.

- Output-ul modelului include atât clasa materialului, cât și poziția 2D a obiectului în imagine, care este ulterior transformată în coordonate 3D prin calibrarea camerei cu ajutorul markerelor ArUco.

4. Calcularea traiectoriei optime a brațului robotic

- Poziția obiectului este transformată într-un punct țintă în spațiul cartezian.
- Se aplică geometria inversă pentru a determina un set de unghiuri ale articulațiilor brațului robotic, astfel încât să se atingă poziția dorită.
- Traiectoria este generată ținând cont de limitele mecanice ale sistemului, iar Jacobianul este utilizat pentru a genera traiectoria.

5. Manipularea obiectelor

- Mișcările brațului sunt animate și testate în MATLAB, folosind transformări omogene și vizualizări 3D pentru a valida corectitudinea.

1.4 Motivație

Motivația principală a acestui proiect a fost interesul pentru domeniul roboticii și dorința de a aborda o problemă de actualitate, și anume reciclarea deșeurilor, prin dezvoltarea unei soluții tehnologice inteligente. Această inițiativă a oferit oportunitatea de a integra și aplica cunoștințe din programare, procesare de imagini, inteligență artificială și electronică.

Proiectul a constat în conceperea unui robot mobil echipat cu un braț robotic, cameră video și senzori specifici, destinat detectării și clasificării automate a materialelor reciclabile. Utilizarea unui sistem de recunoaștere bazat pe procesarea imaginilor, împreună cu marker-ele ArUco pentru poziționare și orientare precisă, a permis integrarea coerentă a percepției cu controlul mișcării. Controlul traiectoriei brațului robotic a fost realizat prin intermediul matricei Jacobiene, care permite transformarea mișcărilor dorite ale efectorului final în comenzi articulate, asigurând o mișcare lină și precisă în mediul de operare.

Prin implementarea acestor tehnologii, s-a realizat un prototip funcțional, care poate contribui la procesele de sortare a deșeurilor, având astfel o potențială aplicabilitate în gestionarea eficientă și sustenabilă a resurselor.

1.5 Structura lucrării

Lucrarea de față este structurată pe 5 capitole, în care sunt prezentate noțiunile necesare pentru atingerea obiectivului propus. În continuare, sunt prezentate, pe scurt, aceste capitole și conținutul lor.

Capitolul 1 este dedicat introducerii temei lucrării, fiind prezentate contextul general, motivația alegerii subiectului, precum și obiectivele urmărite. De asemenea, sunt menționate principalele tehnologii utilizate și aplicațiile în care acestea își pot găsi utilitatea.

Capitolul 2 conține prezentarea noțiunilor fundamentale necesare înțelegerii lucrării, incluzând o introducere în funcționarea rețelelor neurale YOLOv8 pentru detecția obiectelor, utilizarea markerelor ArUco pentru localizare și orientare, precum și metodele modelului geometric folosite pentru descrierea și controlul mișcărilor brațului robotic. De asemenea, sunt prezentate conceptele esențiale legate de servomotoare, evidențiindu-se principiile de funcționare și tipurile de servomotoare. În plus, este detaliat procesul de calibrare a camerei, un pas esențial pentru obținerea unei estimări corecte a poziției și orientării obiectelor în spațiul tridimensional, prin determinarea parametrilor intrinseci și extrinseci ai camerei utilizate în sistemul de viziune artificială.

În capitolul 3 este prezentată partea practică a lucrării, concentrându-se pe implementarea sistemului. Se descrie procesul de adnotare a imaginilor folosite pentru antrenarea rețelei YOLOv8, precum și integrarea componentelor hardware și software pentru a permite robotului să identifice markerii, să-și determine poziția și să execute acțiuni în funcție de informațiile vizuale procesate. În această etapă este abordată și problematica alimentării electrice a sistemului

Capitolul 4 conține rezultatele experimentale obținute în urma implementării sistemului, evaluând performanțele acestuia în diferite scenarii de testare. Sunt analizate eficiența detecției markerelor ArUco, precum și precizia și robustețea rețelei YOLOv8 în identificarea corectă a obiectelor în diverse condiții de iluminare și poziționare. Se discută de asemenea limitele identificate și posibile soluții de îmbunătățire.

În ultimul capitol, capitolul 5, sunt formulate concluziile generale ale lucrării, se evidențiază contribuțiile personale aduse în procesul de dezvoltare și sunt propuse direcții de continuare a proiectului.

2 NOȚIUNI FUNDAMENTALE

2.1 YOLOV8

YOLOv8, dezvoltat de Ultralytics, este un model de detecție a obiectelor, care funcționează prin prezicerea tuturor casetelor de delimitare într-o singură trecere prin rețea, ceea ce îl face eficient și potrivit pentru procesarea în timp real.

YOLOv8 extinde arhitectura populară YOLOv5, aducând îmbunătățiri semnificative. Diferența principală constă în adoptarea unei metode de detecție fără ancore (anchor-free), care accelerează procesul de post-procesare, în special etapa de suprimare non-maximă (NMS). Modelul poate detecta și localiza obiecte în imagini și videoclipuri și este capabil să îndeplinească și sarcini precum clasificarea imaginilor și segmentarea instanțelor.

Modelele din familia YOLOv8 sunt disponibile în mai multe variante, fiecare oferind un echilibru diferit între acuratețe, viteză și dimensiunea modelului. Aceste variante se diferențiază prin anumiți parametri:

- depth_multiple (d) – Controlează numărul de straturi. Valori mai mici (<1) fac modelul mai rapid și mai compact, dar cu o acuratețe mai mică. Valori mai mari (>1) cresc acuratețea, dar încetinesc modelul.
- width_multiple (w) – Controlează numărul de canale. Valori mai mici reduc dimensiunea și cresc viteza, dar pot scădea precizia. Valori mai mari oferă o precizie mai bună, dar necesită mai multe resurse.
- max_channels (mc) – Limitează numărul maxim de canale, pentru a evita ca modelul să devină prea mare sau să supraînvețe.

Tipurile de YOLOv8:

- n (nano) – Cel mai mic și rapid model, dar cu cea mai scăzută acuratețe.
- s (small) – Model mic, cu un bun echilibru între viteză și precizie.

- m (medium) – Model mediu, mai precis decât cel mic, dar mai lent.
- l (large) – Model mare, cu o acuratețe ridicată, dar mai lent.
- xl (extra-large) – Cel mai precis, ideal pentru aplicații care dispun de resurse mari.

Tabel 2.1.1 Tipuri de YOLOv8 [6]

Model Variant	d(depth_multiple)	w(width_multiple)	mc(max_channels)
n	0.33	0.25	1024
s	0.33	0.50	1024
m	0.67	0.75	768
l	1.00	1.00	512
xl	1.00	1.25	512

Tabel 2.1.2 Sarcini și moduri suportate [7]

Model	Task
Yolov8	Detection
Yolov8-seg	Instance Segmentation
Yolov8-pose	Pose/Keypoints
Yolov8-obb	Oriented Detection
Yolov8-cls	Classification

Arhitectura YOLOv8 este compusă din trei componente principale, fiecare având un rol esențial în procesul de detectare a obiectelor: Backbone, Neck și Head:

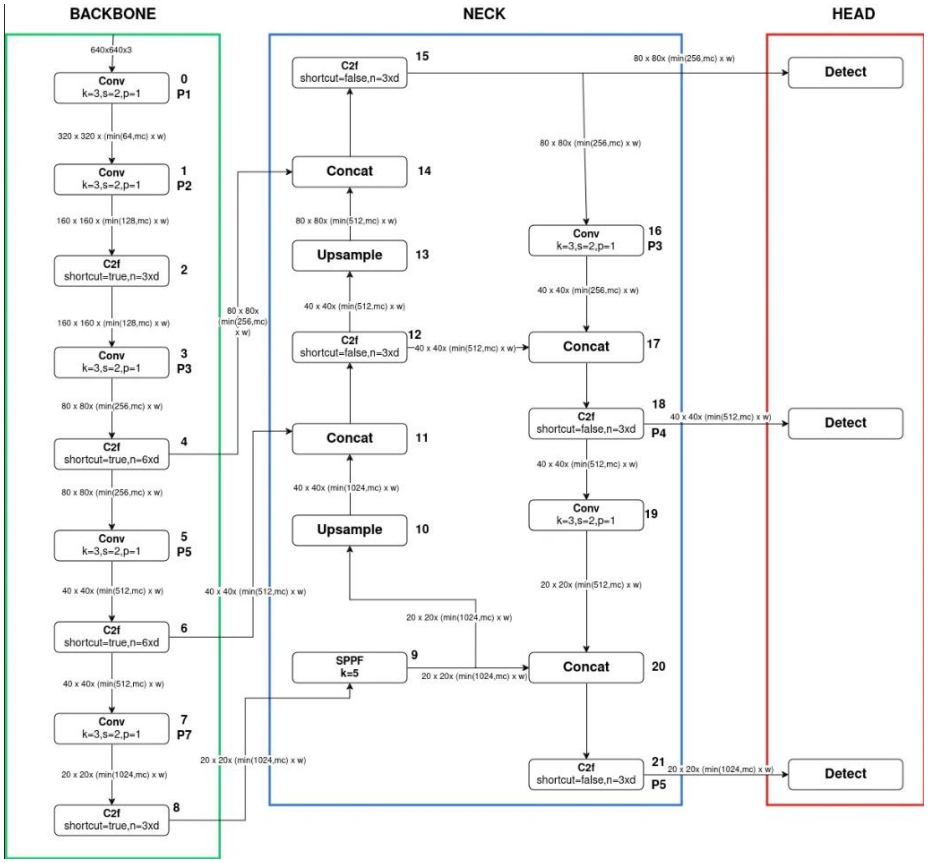


Figura 2.1.1 Arhitectura rețelei de vedere artificială YOLOv8 [6]

2.1.1 Backbone-ul rețelei

Backbone-ul este responsabil pentru extragerea caracteristicilor relevante din imaginea de intrare. În YOLOv8, acest rol este îndeplinit de o arhitectură personalizată bazată pe CSPDarknet53, care utilizează conexiuni parțiale între etape (CSP) pentru a îmbunătăți fluxul de informații între straturi și a reduce redundanța.

Un element esențial în backbone este blocul C2f, care constă într-un bloc convoluțional ale cărui ieșiri sunt împărțite: o parte trece printr-un bloc Bottleneck, iar cealaltă merge direct către un bloc de concatenare.

Blocul SPPF ajută la captarea caracteristicilor la multiple scări fără a pierde informații spațiale.

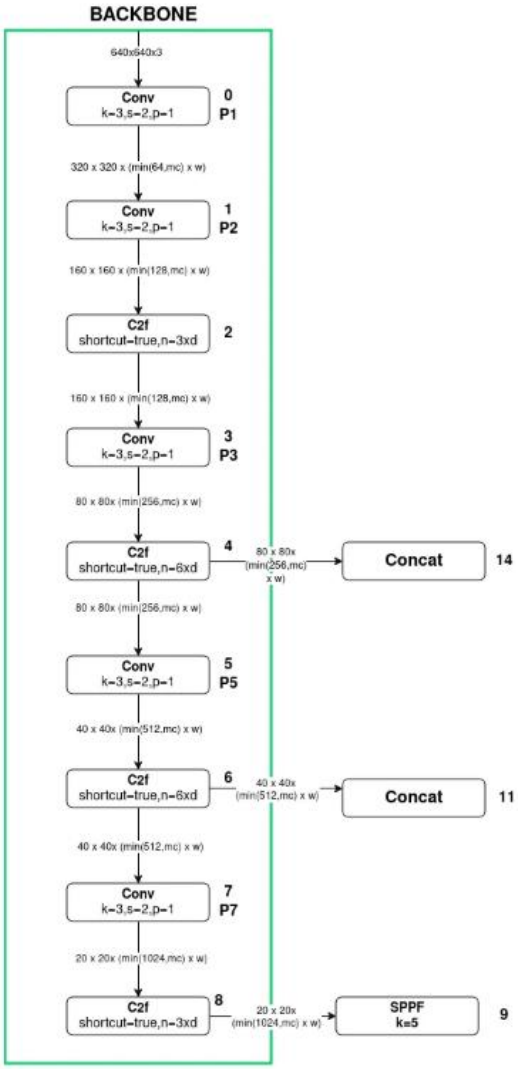


Figura 2.1.2 Backbone-ul rețelei[6]

2.1.2 Neck-ul rețelei

Neck-ul acționează ca o punte între backbone și head, combinând caracteristicile extrase la diferite niveluri pentru a îmbunătăți detecția obiectelor de diverse dimensiuni. YOLOv8 utilizează un modul C2f în neck, care combină caracteristicile semantice de nivel înalt cu informațiile spațiale de nivel scăzut, îmbunătățind performanța detecției, în special pentru obiectele mici.

În neck se folosesc operații de upsampling pentru a mări dimensiunile hărților de caracteristici și blocuri de concatenare pentru a combina informațiile provenite de la diferite niveluri ale backbone-ului.

2.1.3 Head-ul rețelei

Head-ul este responsabil pentru generarea predicțiilor finale, cum ar fi coordonatele cutiilor de delimitare și scorurile de încredere pentru fiecare clasă de obiect. YOLOv8 adoptă o abordare fără ancore (anchor-free), ceea ce înseamnă că modelul prezice direct centrul obiectului, reducând complexitatea post-procesării.

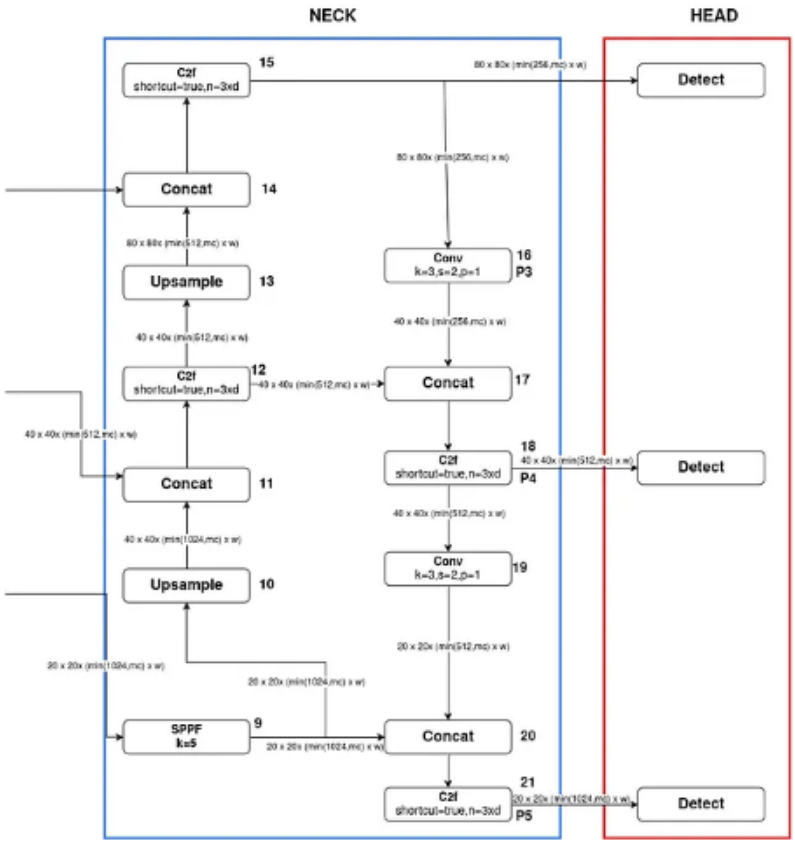


Figura 2.1.3 Neck-ul şi Head-ul rețelei [6]

2.2 ARUCO MARKERS

Estimarea poziției este foarte importantă în multe aplicații de viziune computerizată, precum navigarea roboților, realitatea augmentată și altele. Acest proces se bazează pe identificarea corespondentelor între puncte din mediul real și proiecția lor 2D în imagine.

Un marker ArUco este un marker pătrat sintetic format dintr-o margine neagră groasă și o matrice binară internă care determină identificatorul său (ID-ul). Marginea neagră permite detectarea rapidă a markerului în imagine, iar codificarea binară ajută la identificarea acestuia și permite aplicarea tehnicilor de detectare și corectare a erorilor. Principalul avantaj al acestora este că un singur marker oferă suficiente puncte de corespondență (cele patru colțuri ale sale) pentru a determina poziția camerei.

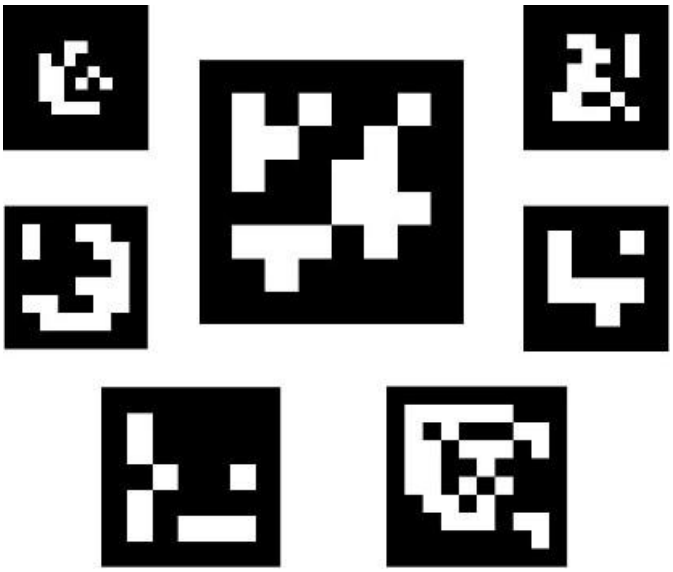


Figura 2.2.1 Markere ArUco [8]

2.2.1 Estimarea poziției și orientării

O aplicație importantă a markerilor ArUco este estimarea poziției și orientării (pose estimation) în raport cu camera. Acest lucru este realizat pe baza poziției colțurilor markerului în imagine și a parametrilor de calibrare ai camerei:

rvec,tvec=cv2.aruco.estimatePoseSingleMarkers(corners,markerLength,cameraMatrix,distCoeff (1.0)

unde:

- **rvec** – vectorul de rotație (care poate fi transformat într-o matrice de rotație *R*);
- **tvec** – vectorul de translație, ce indică poziția markerului față de cameră;
- **cameraMatrix** – matricea intrinsecă a camerei (obținută în urma procesului de calibrare);
- **distCoeffs** – coeficienții de distorsiune optică.

2.2.2 Procesul de detectare

Atunci când o imagine conține markere ArUco, procesul de detecție trebuie să returneze o listă cu markerii detectați. Fiecare marker detectat include:

- Poziția celor patru colțuri în imagine (în ordinea originală).
- ID-ul markerului.
- Estimarea poziției și orientării markerului față de camera ce îl detectează, prin metode de calibrare și estimare a poziției 3D.

2.3 METODELE MODELULUI GEOMETRIC ȘI CINEMATIC

Tabel 2.3.1 Metodele modelului geometric și cinematic [11][12]

Metoda	Relația intrare-ieșire	Observații
Geometria directă	$q \rightarrow \begin{bmatrix} p_E \\ \hat{n}, \hat{o}, \hat{a} \end{bmatrix}$ Unde: p_E - este vectorul de poziție al unui punct E care aparține structurii n, o, a - versorii sistemului de referința asociat punctului E.	Odată cunoscute variabilele generalizate ale articulațiilor, se poate determina atât poziția unui punct arbitrar, cât și orientarea acestuia față de un sistem de referință dat. În general, punctul E desemnează efectorul final, iar sistemul de referință utilizat este cel atașat acestuia. De cele mai multe ori, vectorii calculați sunt exprimați în raport cu sistemul de coordonate global.
Geometria inversa	$\begin{bmatrix} p_E \\ \hat{n}, \hat{o}, \hat{a} \end{bmatrix} \rightarrow q$	În general, rezolvarea problemei se realizează numeric, având la bază modelul cinematic al sistemului.
Reprezentarea configurației	$[q, t_{i+1}^{i}, C, \ldots] \rightarrow$ figura	Aceasta este o funcție compusă care folosește geometria directă pentru a determina pozițiile articulațiilor și elementelor robotului, iar apoi le integrează pentru a genera

		reprezentarea grafică a configurației sale.
Pseudo-animație	$\{figura_{1,\dots,k}\} \rightarrow$ animație	O pseudo-animație constă într-o serie de reprezentări succesive ale diferitelor configurații ale robotului, afișate în ordine pentru a sugera mișcarea.
Calculul variabilelor generalizate pentru sarcina de pick&place	$\begin{cases} q_{i,0} \\ q_{i,\Delta} \\ t = [0, \dots, \Delta] \end{cases} \rightarrow \begin{cases} q_i(t) \\ \dot{q}_i(t) \\ \ddot{q}_i(t) \end{cases}$	Funcția primește ca parametri poziția generalizată inițială, poziția finală și durata execuției sarcinii, iar ca rezultat, determină pentru orice moment din intervalul specificat pozițiile, vitezele și accelerațiile generalizate ale articulațiilor robotului.
Calculul Jacobianului	$\begin{cases} q \\ {}^{o}_{i}T \end{cases} \rightarrow J(q) = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix}$ Unde: q=q(t) este vectorul variabilelor generalizate; ${}^{o}_{i}T$ (i=1,....,n) sunt matricile de transformare omogene(MG); J(q)(6xn) este matricea Jacobian	Jacobianul este un element esențial în analiza cinematică, fiind utilizat pe scară largă în diverse metode. Acesta este un operator matricial care se împarte în două componente: una asociată vitezelor liniare și cealaltă vitezelor unghiulare.
Cinematica directă	$\begin{bmatrix} J(q) \\ \dot{q} \end{bmatrix} \rightarrow \begin{bmatrix} v_e \\ \omega_e \end{bmatrix}$ Unde: $v_e \omega_e$ sunt vitezele liniare ale originii sistemului de referință; respective viteza unghiulară a efectorului.	Odată cunoscute vitezele variabilelor generalizate ale articulațiilor, se pot calcula vectorii vitezelor liniare și unghiulare pentru orice punct al structurii robotului.
Cinematica inversă	$\begin{cases} \begin{bmatrix} v_e \\ \omega_e \end{bmatrix} \\ q \end{cases} \rightarrow \dot{q}$	Funcția primește ca argumente vitezele liniare și unghiulare ale efectorului, precum și valorile actuale ale variabilelor generalizate, iar

		ca rezultat calculează vitezele acestor variabile în momentul respectiv.
Mobilitatea cinematica, volumul de lucru	$\begin{cases} J(q) \\ p_e \end{cases} \rightarrow \mu_c(q)$	Mobilitatea cinematică reprezintă o funcție ce caracterizează abilitatea efectorului de a se mișca în diverse poziții din spațiul cartezian. Zonele în care mobilitatea este diferită de zero definesc volumul de lucru al robotului.

2.4 SERVOMOTOARE

Servomotorul include un circuit de control care oferă feedback despre poziția actuală a axului motorului, permițând astfel controlul precis al mișcării. Dacă motorul este alimentat cu curent continuu, se numește servomotor DC, iar dacă este alimentat cu curent alternativ, se numește servomotor AC. Servomotoarele au, de obicei, un sistem de angrenaje (roți dințate) care le permite să ofere un cuplu mare într-un format compact și ușor.

Servomotoarele sunt evaluate în kg/cm (kilograme pe centimetru). De exemplu, un motor de 6kg/cm poate ridica o greutate de 6 kg dacă aceasta este la 1 cm de axul motorului – dacă greutatea este mai departe, capacitatea de ridicare scade. Poziția motorului este controlată prin impulsuri electrice transmise către circuitul său de comandă.

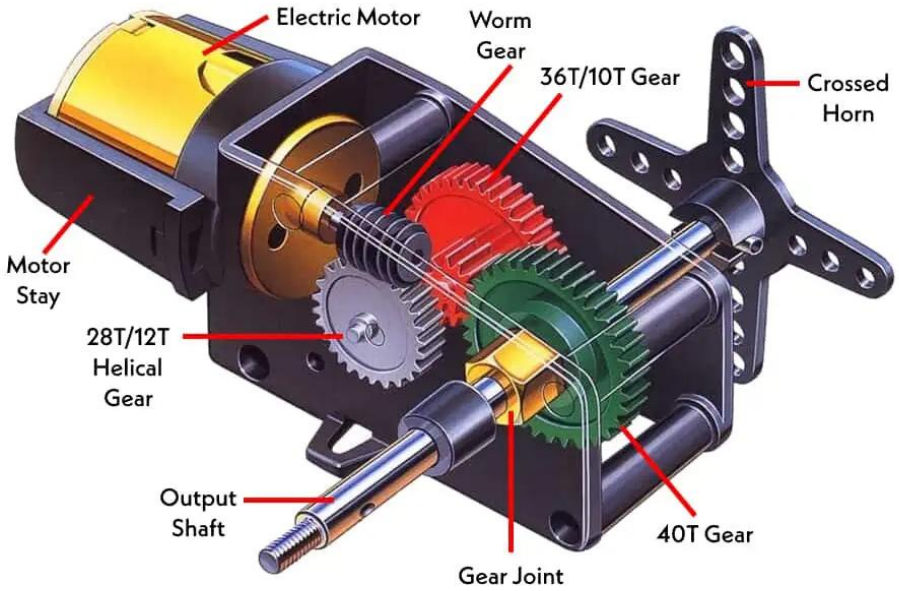


Figura 2.4.1 Construcția unui servomotor [9]

2.4.1 Principiul de funcționare al unui servomotor

Acest tip de sistem funcționează pe baza unei bucle de reglare închisă (feedback loop). Procesul este următorul:

1. Se trimite către servo un semnal de referință ce corespunde unei poziții sau viteze dorite.

2. Encoder-ul citește în timp real poziția curentă a motorului.

3. Semnalul citit este comparat cu semnalul de referință.

4. Diferența dintre cele două generează un semnal de eroare, care este transmis de către controler către motor.

5. Motorul este acționat până când eroarea devine nulă (sau suficient de mică), moment în care poziția este menținută.

2.4.2 Tipuri de servo-motoare

Servomotoare AC:
Funcționează cu curent alternativ. Sunt fiabile și oferă performanță lină la viteze mari și sarcini grele. Folosite frecvent în echipamente industriale precum benzi transportoare, mașini CNC și roboți.

Servomotoare DC:
Funcționează cu curent continuu. Au un design mai simplu și sunt mai ușor de controlat. Sunt accesibile și rapide, ideale pentru jucării, roboți mici și automatizări casnice.

Servomotoare cu Rotație Pozițională:
Cele mai comune; se rotesc la un unghi specific în funcție de semnalul primit. Utilizate în brațe robotice, poziționare de antene și gimbal-uri pentru camere.

Servomotoare cu Rotație Continuă:
Pot roti continuu în ambele direcții, fără unghi fix. Sunt ideale pentru roți și benzi transportoare în roboți.

Servomotoare Liniare:
Transformă mișcarea de rotație în mișcare liniară. Folosite în actuatori industriali și echipamente de precizie.

Servomotoare Fără Perii (Brushless):
Nu au perii, ceea ce înseamnă uzură redusă, eficiență mai mare, viteză mai mare și durată de viață mai lungă. Sunt controlate electronic pentru o mișcare lină și eficientă.

2.5 PARAMETRII DE CALIBRARE A CAMEREI

Parametrii unei camere se împart în trei categorii[18]:

a) Parametrii intrinseci

Aceștia descriu proprietățile geometrice și optice ale camerei, independente de poziția acesteia în spațiu. Matematic, acești parametri sunt organizați într-o **matrice de calibrare internă** (matrice intrinsecă), notată de regulă cu K:

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1.1)$$

Unde:

f_x, f_y: distanțele focale exprimate în pixeli pe axele X si Y

c_x, c_y: coordonatele punctului principal (centrul optic)

γ-factorul de skew (care descrie neortogonalitatea axelor X și Y ale imaginii; în majoritatea cazurilor este 0)

Acești parametri depind de senzorul camerei și de lentilele utilizate.

b) Parametrii extrinseci

Parametrii extrinseci definesc poziția și orientarea camerei față de un sistem de referință global (de exemplu, un sistem de coordonate al unei scene sau al unui obiect). Aceștia sunt reprezentați prin:

- o **matrice de rotație** R, care descrie orientarea camerei;
- un **vector de translație** t, care descrie poziția camerei în spațiu.

Împreună, aceștia transformă punctele din sistemul global de coordonate în sistemul camerei.

c) Coeficienții de distorsiune

În practică, majoritatea camerelor introduc deformări în imagine, cauzate de imperfecțiuni ale lentilelor. Cele mai comune tipuri de distorsiuni sunt:

- **Distorsiunea radială** (liniile drepte apar curbate), modelată de coeficienții k_1, k_2, k_3
- **Distorsiunea tangențială** (poziția colțurilor este afectată de înclinarea lentilei), modelată de coeficienții p_1, p_2

Aceste distorsiuni trebuie eliminate pentru ca imaginea să reflecte corect geometria scenei.

3 IMPLEMENTARE

3.1 DESCRIEREA PROBLEMEI

Primul pas în proiectarea oricărui sistem constă în identificarea clară a sarcinilor care trebuie realizate pentru atingerea obiectivelor propuse. Aceste sarcini sunt, în general, complexe și necesită o abordare modulară, astfel încât sistemul să fie împărțit în componente funcționale mai mici, fiecare responsabilă pentru o subproblemă bine definită. În cazul de față, sistemul propus constă într-un robot mobil dotat cu un braț robotic, care utilizează o cameră montată pentru identificarea și clasificarea materialelor (plastic, carton, aluminiu, sticlă) și determinarea poziției obiectelor folosind markere ArUco.

Obiectivul principal al sistemului este detecția materialelor reciclabile, poziționarea corectă în raport cu obiectul, manipularea acestuia și sortarea corespunzătoare.

Detecția obiectelor reprezintă una dintre cele mai importante sarcini ale sistemului. Aceasta se realizează folosind o rețea neuronală convoluțională de tip YOLOv8, aleasă pentru capacitatea sa de a realiza inferență rapidă în timp real și de a oferi atât clasificarea obiectului cât și poziția acestuia în imagine sub forma unui bounding box. Pentru antrenarea modelului, a fost construit un set de date personalizat, alcătuit din imagini care conțin obiecte fabricate din patru tipuri de materiale: plastic, sticlă, carton și aluminiu. Acest set de date a fost etichetat manual corespunzător, astfel încât YOLOv8 să poată învăța să distingă între tipurile de materiale.

Pentru a putea manipula eficient obiectele, nu este suficientă doar detecția 2D. De aceea, sistemul folosește markere ArUco, care permit determinarea poziției și orientării obiectului în raport cu camera, prin tehnici de estimare a poziției 3D (pose estimation). Astfel, robotul poate să se orienteze corect și să manipuleze obiectul cu precizie.

Pe baza datelor primite de la rețeaua neuronală și de la markerii ArUco, sistemul calculează pașii necesari pentru:

- Deplasarea platformei mobile către obiect.
- Poziționarea brațului robotic în direcția corectă.
- Manipularea și sortarea obiectului în funcție de materialul detectat.

Deciziile sunt implementate pe un Raspberry Pi 5, care funcționează ca unitate centrală de procesare (master). Acesta rulează modelul YOLOv8 pentru procesarea imaginilor, interpretează markerii ArUco și generează comenzile necesare pentru controlul robotului.

Întregul sistem hardware este coordonat de Raspberry Pi, care gestionează atât procesarea de înalt nivel (vedere artificială, inferență, logică de control), cât și controlul în timp real al componentelor fizice. Semnalele PWM necesare pentru acționarea motoarelor de deplasare și a servomotoarelor brațului robotic sunt transmise prin intermediul unei plăci de dezvoltare PCA9685, conectată la Raspberry Pi.

Această arhitectură permite un control eficient și precis al întregului sistem robotic, eliminând nevoia unui microcontroler suplimentar, precum Arduino.

Sistemul robotic este alcătuit din următoarele componente principale:

- Platformă mobilă cu șenile, propulsată de motoare DC;
- Braț robotic acționat prin servomotoare, utilizat pentru operațiuni de tip *pick and place*;
- Cameră RGB, montată pe platformă, utilizată pentru captarea imaginilor din mediul real în vederea detecției obiectelor și identificării markerilor ArUco;
- Markeri ArUco, plasați pe obiectele reciclabile, care permit localizarea spațială precisă a acestora în raport cu sistemul de coordonate al robotului;
- Placă de dezvoltare PCA9685, utilizată pentru generarea semnalelor PWM necesare controlului servomotoarelor și motoarelor;
- Raspberry Pi care funcționează ca unitate centrală, rulând modelul YOLOv8 pentru clasificare vizuală și efectuând calculele necesare pentru controlul cinematic al brațului (inclusiv Jacobianul).

3.2 MODELAREA SISTEMULUI

Un model reprezintă o formă abstractă, simplificată sau formalizată a unui sistem real, utilizată pentru a înțelege, analiza, simula sau controla comportamentul acestuia. În contextul

proiectului propus, modelarea sistemului are ca scop optimizarea funcționării robotului mobil cu braț robotic, înainte de construirea și testarea efectivă a componentei fizice.

Modelul permite testarea performanțelor și ajustarea parametrilor pentru ca sistemul să funcționeze optim în sarcina de detecție, poziționare și sortare a materialelor reciclabile. Primul pas constă în realizarea unei diagrame bloc simplificate, care evidențiază arhitectura generală și fluxul de informații, fără a intra în detalii tehnice despre fiecare modul.

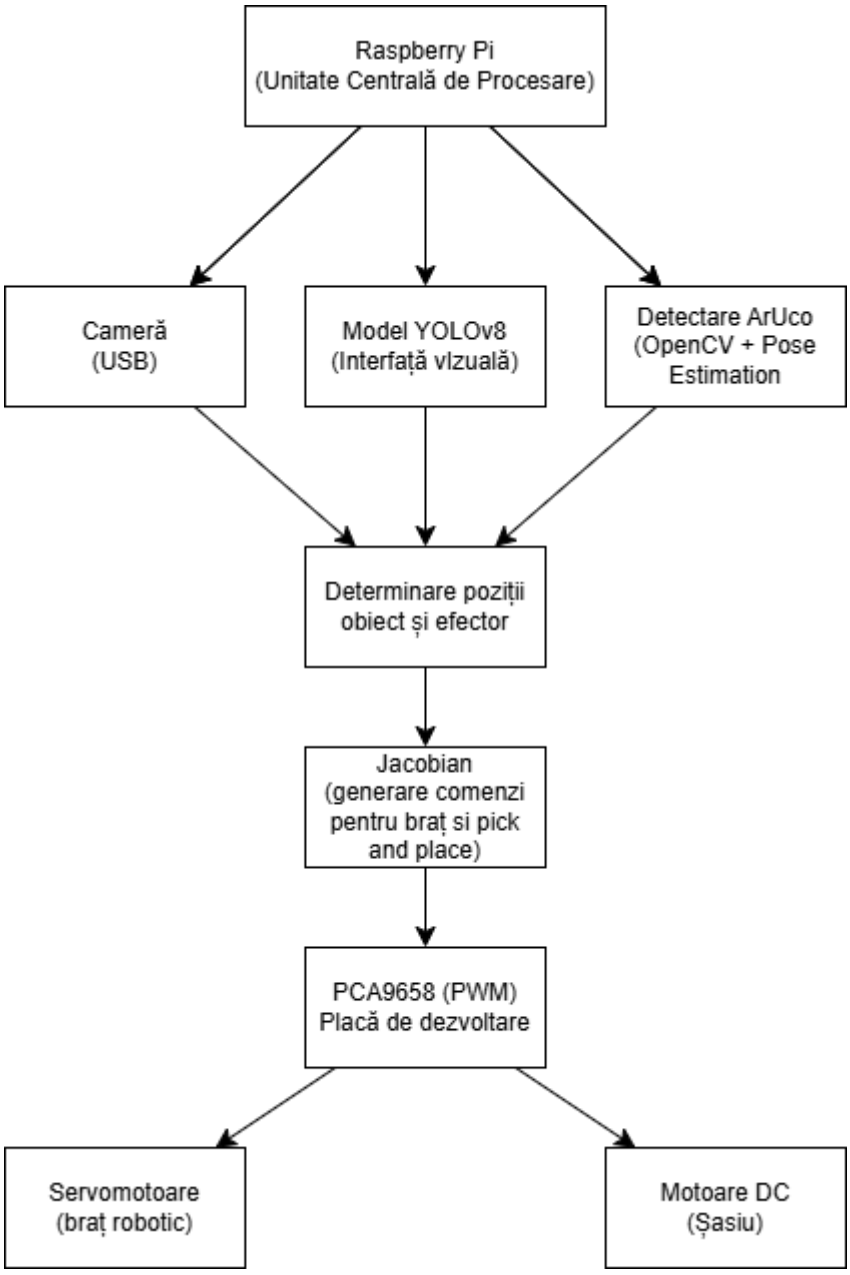


Figura 3.2.1 Diagrama sistemului

Pentru a modela corect comportamentul sistemului, este necesară cunoașterea următorilor parametri ai componentelor:

- Parametrii camerei: pentru calibrare
- Geometria brațului robotic: lungimea segmentelor, limitele de rotație.

Modelarea acestor componente va fi realizată în capitolele următoare, pentru a putea simula comportamentul real cât mai precis.

3.3 ANTRENAREA SI EVALUAREA MODELULUI YOLOV8

Modelul YOLOv8n, dezvoltat de Ultralytics, a fost ales datorită timpului scurt de procesare și a dimensiunii reduse, comparativ cu celelalte variante ale arhitecturii YOLOv8. Chiar dacă este o versiune compactă („nano”), modelul este capabil să detecteze obiecte cu o acuratețe ridicată, fiind ideal pentru implementare pe platforme embedded sau pe sisteme cu resurse hardware limitate.

Obiectivul principal al rețelei este recunoașterea în timp real a materialelor reciclabile, utilizând imagini captate de o cameră video. Astfel, s-a realizat un set de date personalizat, alcătuit din patru clase de materiale:

- PET-uri (plastic transparent),
- doze de aluminiu,
- sticle,
- carton.

Dintr-un set mai mare de date numit TrashBox [9], s-au selectat aproximativ **130 de imagini pentru fiecare clasă**, rezultând un total de aproximativ **500 de imagini**. Acestea surprind obiectele din unghiuri variate, în diverse condiții de iluminare și fundal, pentru a asigura generalizarea modelului.

Pentru etichetarea obiectelor din imagini s-a utilizat platforma **CVAT (Computer Vision Annotation Tool)** [10], care permite desenarea manuală a bounding box-urilor și exportul în format compatibil cu YOLO.

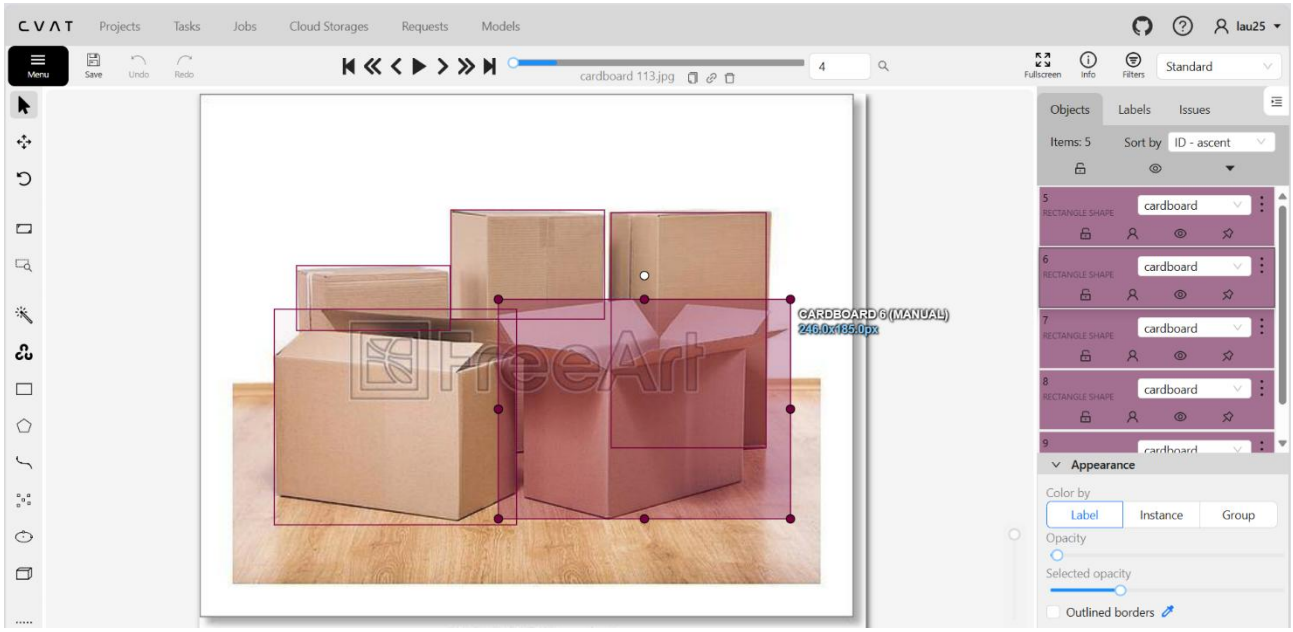


Figura 3.3.1 Etichetarea manuală a imaginilor

Totuși, performanța modelului a fost afectată în anumite situații, în special din cauza rezoluției reduse a imaginilor utilizate în procesul de antrenare. Din această cauză, în unele cazuri, modelul a generat predicții eronate. De asemenea, s-au observat confuzii frecvente între clasele plastic și aluminiu, mai ales în condiții de iluminare slabă sau neuniformă. Un alt factor care a influențat negativ acuratețea a fost reprezentat de apariția PET-urilor deformate, strivite sau murdare, care nu au fost suficient reprezentate în setul de date de antrenare. Acest lucru a dus la o dificultate crescută în recunoașterea corectă a acestor obiecte.

Rezultatul acestei inferențe constă într-o listă de bounding box-uri (casete de încadrare), reprezentate sub forma unor dreptunghiuri descrise prin coordonatele colțurilor opuse:

$$(x_{min}^{bbox}, y_{min}^{bbox}), (x_{max}^{bbox}, y_{max}^{bbox})$$

care definesc regiunile din imagine în care au fost detectate deșeuri reciclabile. Fiecare bounding box este asociat cu o etichetă de clasă corespunzătoare unui tip de material reciclabil, conform fișierului YAML utilizat în antrenarea modelului: 0 - carton, 1 - sticlă, 2 - plastic, 3 – aluminiu.

În figura 3.3.2 sunt prezentate câteva dintre imaginile utilizate în cadrul setului de date.

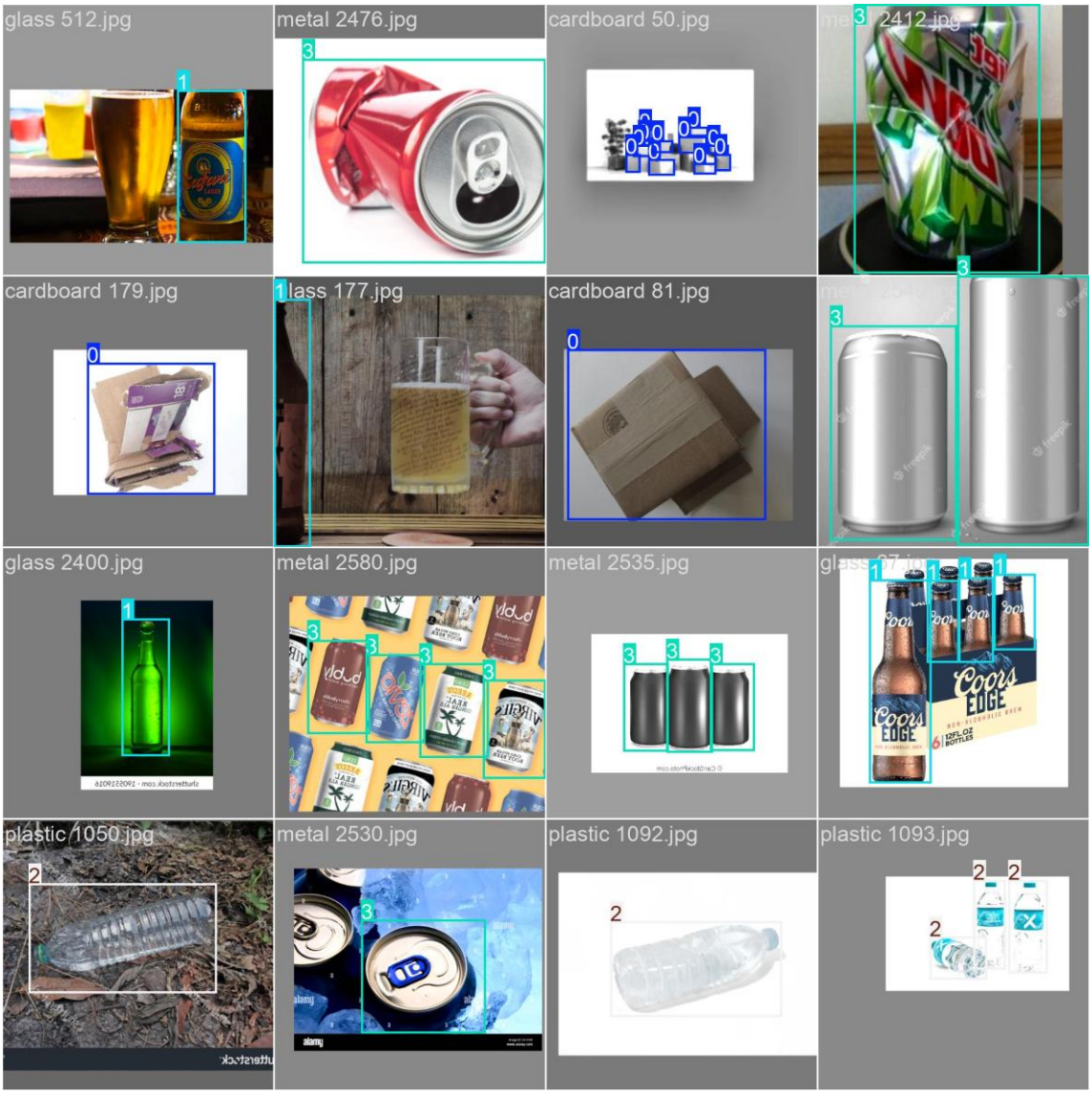


Figura 3.3.2 Setul de date

Antrenarea rețelei a fost realizată în două etape, pentru a compara performanțele în funcție de numărul de epoci:

- **Antrenare A:** 20 de epoci
- **Antrenare B:** 100 de epoci

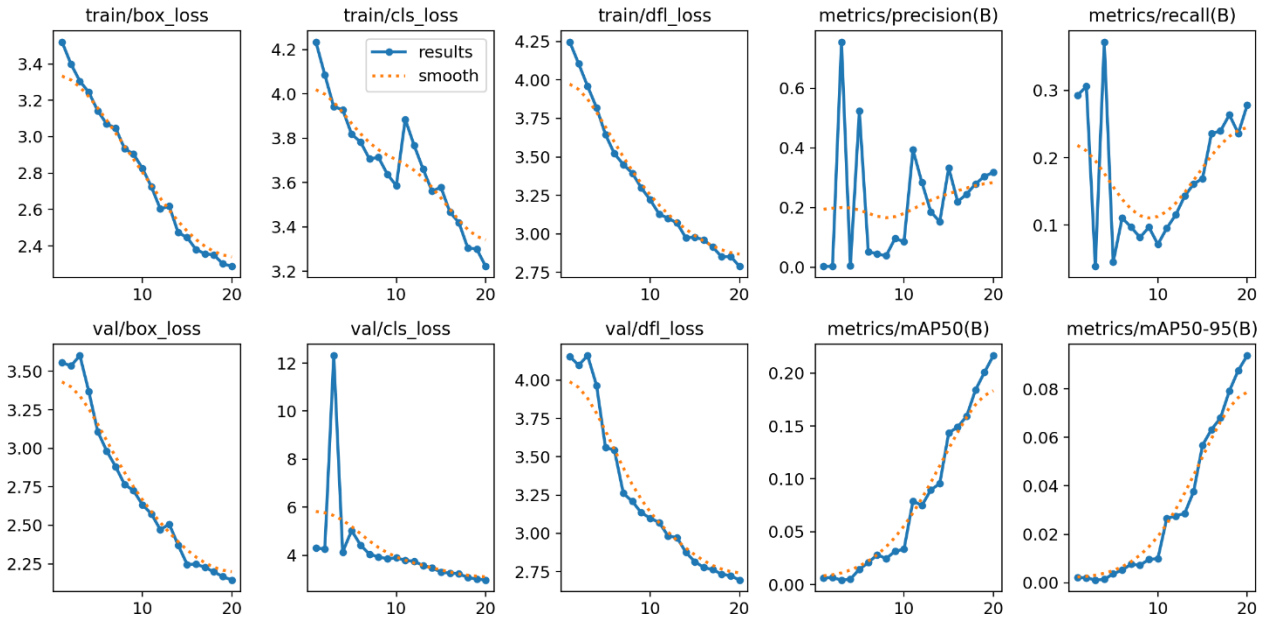


Figura 3.3.3 Rezultate 20 de epoci

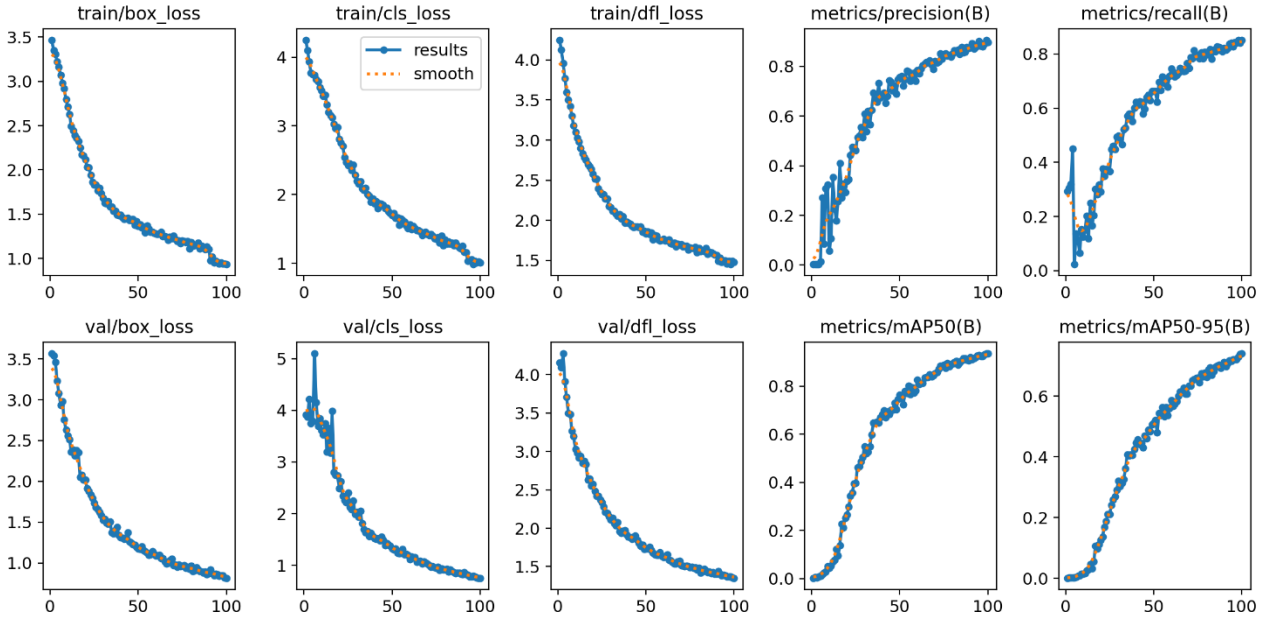


Figura 3.3.4 Rezultate 100 de epoci

În urma analizării rezultatelor obținute după antrenarea modelului timp de 20 de epoci (Figura 3.3.3) și respectiv 100 de epoci (Figura 3.3.4), se observă îmbunătățiri semnificative în performanța generală a rețelei. Pierderile (loss-urile) pentru antrenare și validare — precum box_loss, cls_loss și dfl_loss — scad considerabil mai mult după 100 de epoci, indicând o învățare mai profundă și mai stabilă.

Metricile de performanță, precum **precizia**, **recall-ul** și **mAP-ul,** cresc constant odată cu numărul de epoci. Dacă după 20 de epoci valorile acestor metrici erau scăzute și instabile, la 100 de epoci ele ating niveluri mult mai ridicate și stabile (de exemplu, mAP50 ajunge de la ~0.2 la ~0.85).

Aceste rezultate confirmă că un număr mai mare de epoci contribuie esențial la îmbunătățirea capacității modelului de a detecta și clasifica corect obiectele, fără a prezenta semne evidente de suprapotrivire. Prin urmare, antrenarea extinsă duce la o performanță superioară atât pe datele de antrenament, cât și pe cele de validare.

3.4 CALIBRAREA CAMEREI

Calibrarea camerei este procesul prin care se determină toți parametrii, descriși anterior în capitolul 2.5 Parametrii de calibrare a camerei, intrinseci, extrinseci și coeficienții de distorsiune. Scopul este de a putea reconstrui geometria 3D a scenei și de a corecta imaginile captate. În această secțiune, se prezintă procesul complet de calibrare a unei camere folosind imagini cu o tablă de șah, urmat de utilizarea coeficienților obținuți pentru detectarea și poziționarea markerilor ArUco.



Figura 3.4.1 Cameră Canyon C3 [13]

Tabel 3.4.1 Caracteristici cameră web[13]

Rată maximă de cadre	30fps
Interfață	USB 2.0
Adâncime(mm)	57,8
Înălțime(mm)	46,5
Lățime(mm)	62,2
Rezoluție video maximă	1280x720

Una dintre cele mai utilizate metode este metoda lui **Zhang**, care presupune utilizarea unui model plan cu geometrie cunoscută, de regulă, o tablă de șah. Pașii principali sunt:

1. **Captarea mai multor imagini** ale modelului de calibrare (12 imagini din unghiuri diferite).
2. **Detectarea colțurilor** tablei de șah în fiecare imagine (7x10, 7 colțuri pe lățime si 10 pe lungime)
3. **Asocierea colțurilor detectate** cu pozițiile cunoscute din spațiul real (2D vs. 3D).
4. **Estimarea parametrilor camerei** prin minimizarea erorii de reproiecție — diferența dintre poziția proiectată și poziția reală a colțurilor în imagine.
5. **Calculul erorii medii de calibrare**, care oferă un indicator al preciziei procesului.

Proiecția unui punct 3D din lume pe senzorul camerei se face conform relației:

$$s * p = K * [R|t]^* \mathrm{P} \quad (1.2)$$

unde:

- P: punctul 3D în coordonate globale,
- p: punctul 2D proiectat pe imagine,
- K: matricea intrinsecă,
- [R|t]: matricea extrinsecă,
- s: factor de scalare (proiecție în planul imaginii).

Trebuie să se precizeze rezoluția camerei web (720p în acest caz) și dimensiunea unui pătrat din tablă (2cm).

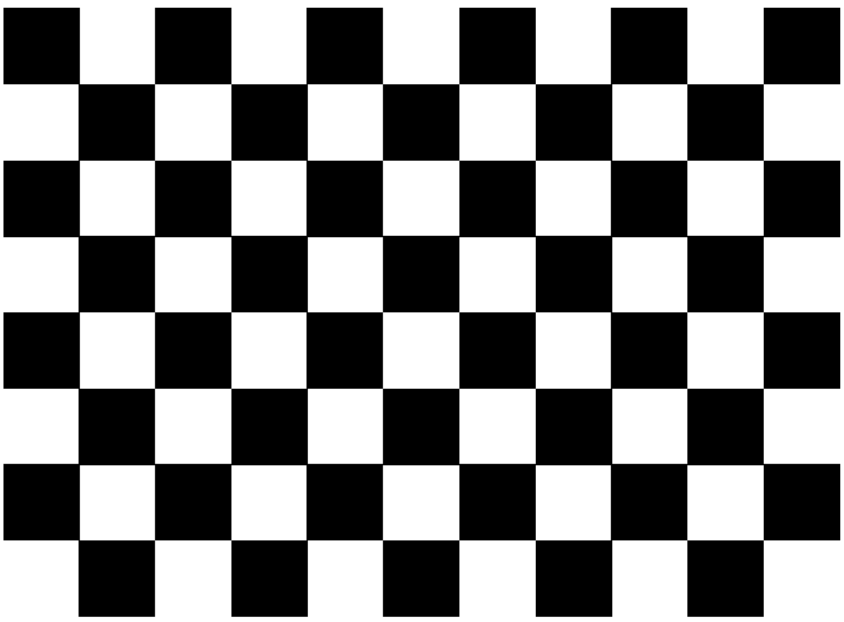


Figura 3.4.2 Tablă de șah [14]

Pentru fiecare imagine, se detectează colțurile interioare ale tablei de șah.

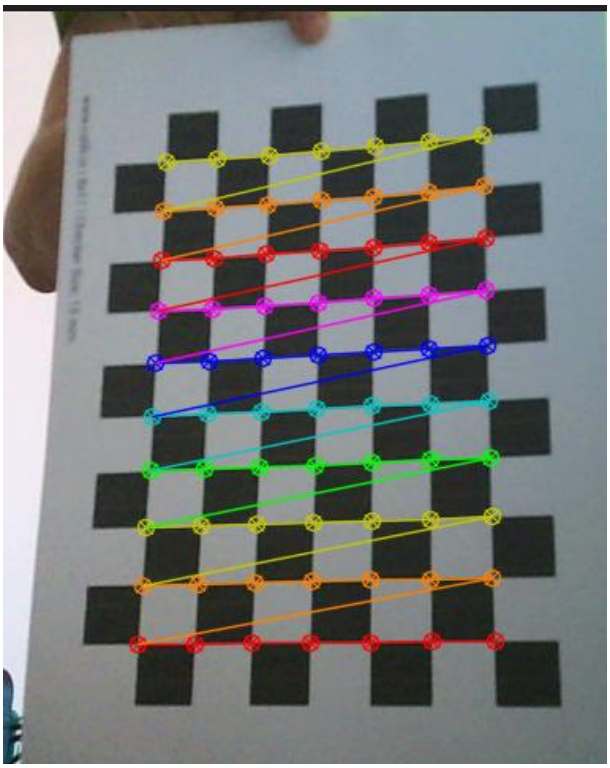


Figura 3.4.3 Colțuri detectate

Se folosesc punctele detectate pentru a calcula matricea camerei și coeficienții de distorsiune, conform relației (1.1).

Se precizează dimensiunea unui marker (10cm), iar apoi se detectează markerul în cadrul capturat de cameră.

Pentru fiecare marker detectat, se estimează vectorii de rotație (orientare) și translație (poziție) în raport cu camera.[17]

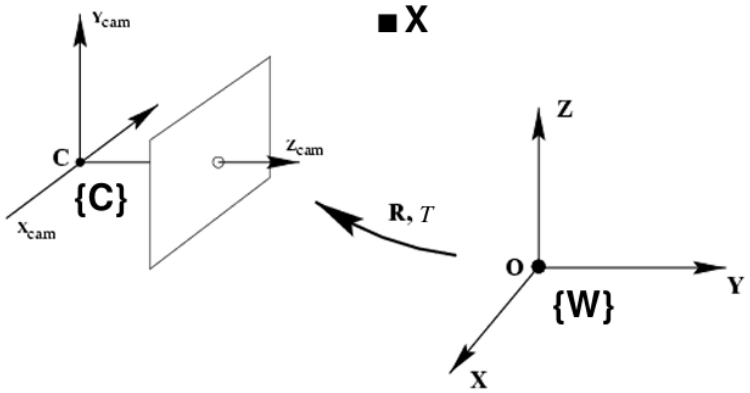


Figura 3.4.4 Translația si rotația camerei [18]

3.5 MODELUL GEOMETRIC SI CINEMATIC AL MANIPULATORULUI

Modelul geometric al manipulatorului constituie o reprezentare matematică formală, esențială în analiza și proiectarea sistemelor robotice. Acesta este structurat în jurul a trei componente fundamentale: parametri, metode și ipoteze.[11][12]

Modelul funcționează ca un ansamblu de metode care operează pe baza valorilor definite în colecția de parametri, ce includ atât constante, cât și variabile caracteristice structurii robotului. Corectitudinea și fidelitatea modelului în raport cu realitatea sunt determinate de ipotezele asumate. Mai exact, predicțiile generate de metode sunt valide doar în condițiile în care ipotezele sunt respectate, iar valorile parametrilor sunt cunoscute cu precizie.

Modelul geometric este utilizat în principal pentru două scopuri:

1. Vizualizarea configurațiilor robotului, prin generarea unor reprezentări grafice care ilustrează dispunerea cuplelor și a elementelor de legătură dintre acestea, oferind o imagine intuitivă a structurii robotice;

2. Fundamentarea modelelor de ordin superior – cum sunt modelele cinematice, dinamice și cele de tip prototip virtual – prin furnizarea unei baze matematice riguroase necesare dezvoltării metodelor aferente acestora.

Pentru elaborarea modelului geometric al manipulatorului, se adoptă următoarele ipoteze, menite să simplifice analiza și să delimiteze domeniul de aplicabilitate al rezultatelor:

1. Configurația robotului este considerată atemporală, adică nu depinde de timp. Aceasta este tratată ca o stare statică, similară unei imagini sau reprezentări grafice, asupra căreia timpul nu are nicio influență.

2. Mișcarea nu este luată în considerare, prin urmare, nu se includ în analiză vitezele liniare sau unghiulare ale cuplelor ori ale elementelor componente.

3. Se neglijează orice interacțiune cu mediul exterior sau interior. Astfel, nu se consideră influențe externe, precum forțele gravitaționale, și nici efecte interne, cum ar fi momentele generate de actuatori sau alte surse de acționare.

4. Parametrii structurii sunt considerați perfect cunoscuți, fiind determinați cu exactitate, fără incertitudini sau erori de măsurare.

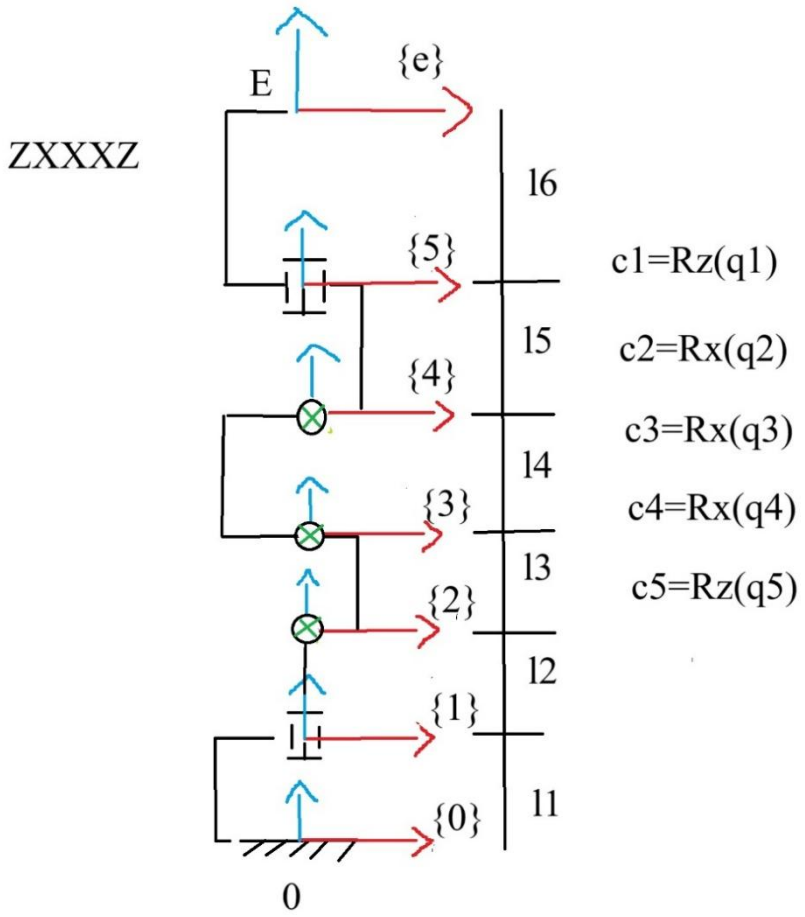


Figura 3.5.1 Desen 5DOF

Parametrii: {n, t_{i+1}^{i}, C, q}

Unde:

-n - numărul elementelor(cuplelor)

Deși robotul analizat este echipat cu 6 cuple, doar primele 5 sunt utilizate în cadrul modelului geometric. Ultima cuplă este destinată exclusiv acționării efectorului final (prindere/desprindere), fără a influența poziția sau orientarea acestuia în spațiul de lucru. Din acest motiv, în modelul simbolic și în simulare, această cuplă nu a fost inclusă.

-t_{i+1}^{i}- geometria elementului

$$t_1^0 = [0,0,l_1]^T$$
$$t_2^1 = [0,0,l_2]^T$$
$$t_3^2 = [0,0,l_3]^T$$
$$t_4^3 = [0,0,l_4]^T$$
$$t_5^4 = [0,0,l_5]^T$$
$$t_e^5 = [0,0,l_6]^T$$

Unde:

-l2=20mm

-l3=105mm

-l4=145mm

-l5=70mm

-l6=110mm

-C - tipul cuplei

C = $\{R_Z, R_X, R_X, R_X, R_X, R_Z\}$

Tipul cuplei i:

-Rotatie pe axa Z: ${}^{1-i}_{i}T = \begin{bmatrix} cos_{qi} & -sin_{qi} & 0 & t_x \\ sin_{qi} & cos_{qi} & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} := {}^{i-1}_{i}T_{Rz}$ (2.0)

-Rotatie pe axa X: ${}^{1-i}_{i}T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & cos_{qi} & -sin_{qi} & t_y \\ 0 & sin_{qi} & cos_{qi} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} : {}^{i-1}_{i}T_{Rx}$ (2.1)

-q – variabila generalizată din cuple (pozițiile unghiulare ale cuplelor sunt cunoscute)

Q=$[q_1, q_2, q_3, q_4, q_5]^T$

3.5.1 Geometria directă

Geometria directă se referă la determinarea poziției și orientării efectorului final (end-effector) al unui robot în funcție de un set de parametri articulați (unghiuri, translații). Cu alte cuvinte, dacă știm cum sunt poziționate articulațiile robotului, vrem să aflăm unde este vârful (efectorul) în spațiu. Pentru un manipulator serial, poziția finală se determină prin înmulțirea succesivă a matricilor de transformare asociate fiecărei articulații și fiecărui segment de braț. Fiecare matrice de transformare $T_{i(i-1)}$ descrie cum se transformă coordonatele din cadrul sistemului de referință local al unei articulații față de cel anterior.

Date inițiale: {n, t^i_{i+1}, C, q} → ${}^{i-1}_{i}T$ → ${}^{0}_{i}T$ → Rezultat: ${}^{0}_{e}T$ → $\begin{cases} p_e \\ \hat{n}\ \hat{o}\ \hat{a} \end{cases}$

Calculul operatorilor omogeni de transformare de la sistemul de referință {i+1} la sistemul de referință {i}, realizat prin utilizarea FRT:

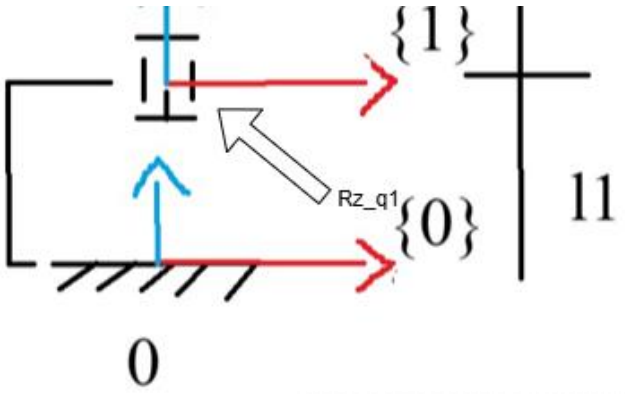


Figura 3.5.2 Baza-cupla1

Prima distanță – cea dintre sol (sau baza fixă) și prima cuplă – nu a fost inclusă în model (cele două repere au origini comune), deoarece s-a considerat direct lanțul cinematic începând cu prima articulație activă. Astfel, toate distanțele și transformările au fost definite din cuplă în cuplă, conform convenției standard pentru analiza cinematică a roboților.

Rotatie pe axa Z: : ${}^{0}_{1}T = \begin{bmatrix} cos_{q1} & -sin_{q1} & 0 & 0 \\ sin_{q1} & cos_{q1} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

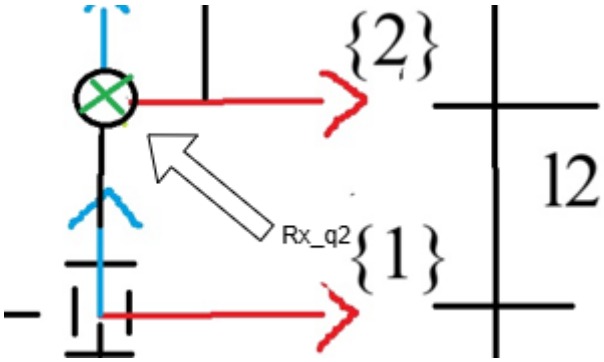


Figura 3.5.3 Cupla1-Cupla2

Rotatie pe axa X: : ${}^{1}_{2}T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos_{q2} & -sin_{q2} & 0 \\ 0 & sin_{q2} & cos_{q2} & l_2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

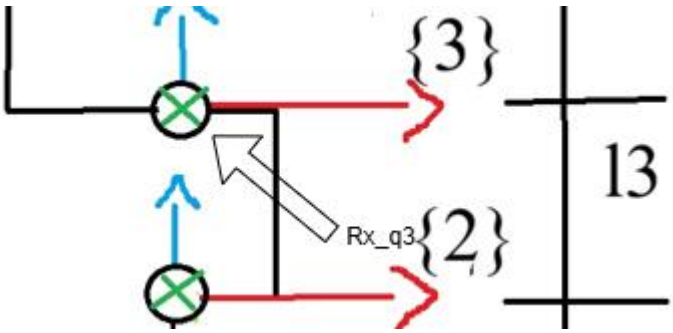


Figura 3.5.4 Cupla2-Cupla3

Rotatie pe axa X: : ${}^{2}_{3}T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos_{q3} & -sin_{q3} & 0 \\ 0 & sin_{q3} & cos_{q3} & l_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

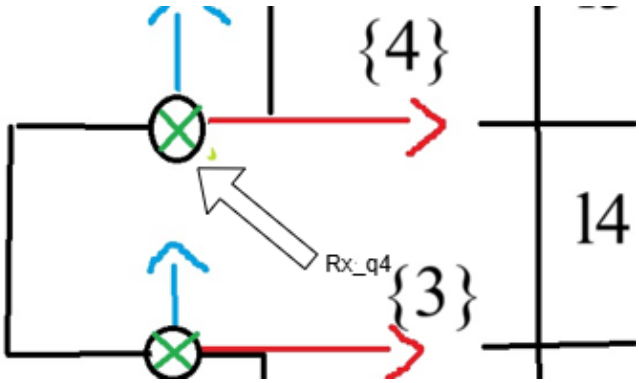


Figura 3.5.5 Cupla3-Cupla4

Rotatie pe axa X: : ${}^{3}_{4}T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos_{q4} & -sin_{q4} & 0 \\ 0 & sin_{q4} & cos_{q4} & l_4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

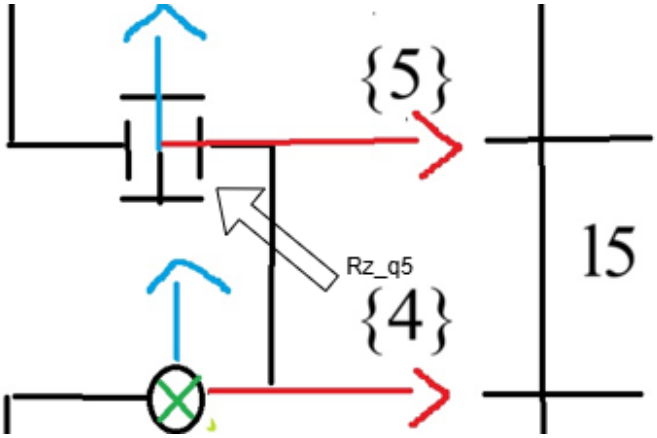


Figura 3.5.6 Cupla4-Cupla5

Rotatie pe axa Z: : ${}^{4}_{5}T = \begin{bmatrix} cos_{q5} & -sin_{q5} & 0 & 0 \\ sin_{q5} & cos_{q5} & 0 & 0 \\ 0 & 0 & 1 & l_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

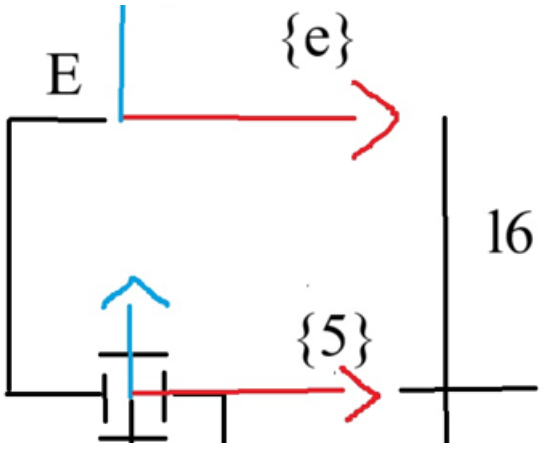


Figura 3.5.7 Cupla5-Efector

${}^{5}_{e}T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & l_6 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Punctele $p_0, p_1, p_2, p_3, p_4, p_5$ reprezintă pozițiile succesive ale articulațiilor și ale efectorului final (end-effector) ale brațului robotic, calculate în raport cu sistemul de coordonate global. Fiecare punct este obținut prin înmulțirea unui vector de poziție inițial P_0=[0;0;0;1](scris în coordonate omogene) cu matricea de transformare omogenă corespunzătoare fiecărei articulații. Astfel, p_0 se obține din înmulțirea lui P_0 cu matricea ${}^0_1T$, care descrie poziția și orientarea primei articulații, p_1 se obține din ${}^0_1T * {}^1_2T * P_0$, adică poziția celei de-a doua articulații față de baza robotului, și așa mai departe până la $p_5(p_e)$, care este poziția efectorului final, obținută prin compunerea tuturor transformărilor ${}^0_1T * {}^1_2T * {}^2_3T * {}^3_4T * {}^4_5T * {}^5_eT * P_0$.

$$ {}^0_eT = $$

$$\begin{bmatrix} cos(q_1)*cos(q_5)-\sigma_1*sin(q_1)*sin(q_5) & -cos(q_1*sin(q_5)-\sigma_1*cos(q_5)*sin(q_1) & \sigma_3*sin(q_1) & 5*sin(q_1)*\sigma_2 \\ cos(q_5)*sin(q_1)+\sigma_1*cos(q_1)*sin(q_5) & \sigma_1*cos(q_1)*cos(q_5)-sin(q_1)*sin(q_5) & -\sigma_3*cos(q_1) & -5*cos(q_1)*\sigma_2 \\ \sigma_3*sin(q_5) & \sigma_3*cos(q_5) & \sigma_1 & 180*\sigma_1+145*cos(q_2+q_3)+105*cos(q_2)+20 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Unde:

-σ_1=cos $(q_2 + q_3 + q_4)$

-σ_2=36*$\sigma_3 + 29 * sin(q_2 + q_3) + 21 * sin(q_2)$

-$\sigma_3 = sin(q_2 + q_3 + q_4)$

De unde rezultă faptul că:

$$p_e = \begin{bmatrix} 5*sin(q_1)*\sigma_2 \\ -5*cos(q_1)*\sigma_2 \\ 180*\sigma_1+145*cos(q_2+q_3)+105*cos(q_2)+20 \end{bmatrix}$$

$$o_e =$$

$$\begin{bmatrix} cos(q_1)*cos(q_5)-\sigma_1*sin(q_1)*sin(q_5) & -cos(q_1*sin(q_5)-\sigma_1*cos(q_5)*sin(q_1) & \sigma_3*sin(q_1) \\ cos(q_5)*sin(q_1)+\sigma_1*cos(q_1)*sin(q_5) & \sigma_1*cos(q_1)*cos(q_5)-sin(q_1)*sin(q_5) & -\sigma_3*cos(q_1) \\ \sigma_3*sin(q_5) & \sigma_3*cos(q_5) & \sigma_1 \end{bmatrix}$$

În cazul când toate cuplele sunt 0 ($q_1 = 0, q_2 = 0, q_3 = 0, q_4 = 0, q_5 = 0$), robotul se află în poziție verticală, iar p_e și o_e devin :

$$p_e = \begin{bmatrix} 0 \\ 0 \\ 450 \end{bmatrix}$$

Unde, 450mm semnifică toate brațele adunate:

$l_2 + l_3 + l_4 + l_5 + l_6$ = 20 + 105 + 145 + 70 + 110 = 450mm (se verifică)

$$o_e = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3.5.2 Reprezentarea configurației

Adesea, este necesară obținerea reprezentării grafice a unei configurații specifice a robotului. O astfel de reprezentare poate avea forme variate și grade diferite de complexitate, începând de la abstractizări simple, cum ar fi cercuri și segmente (pentru cuple și elemente), și

ajungând până la modele tridimensionale detaliate, similare cu ansamblurile CAD, care includ caracteristici precise ale formei elementelor și cuplelor.

Toate aceste abstractizări sunt definite în raport cu propriile sisteme de referință locale, iar metoda de reprezentare a configurației implică transformarea acestor sisteme locale în sistemul de referință global, obținut prin aplicarea metodei de geometrie directă.

Pentru realizarea reprezentării, se utilizează punctele $p_0, p_1, p_2, p_3, p_4, p_5$ folosite la geometria directă, care servesc la construirea segmentelor ce unesc elementele componente, facilitând astfel vizualizarea structurii robotului.

Date inițiale: [q,t_{i+1}^{i},C,...] → Rezultat : figura

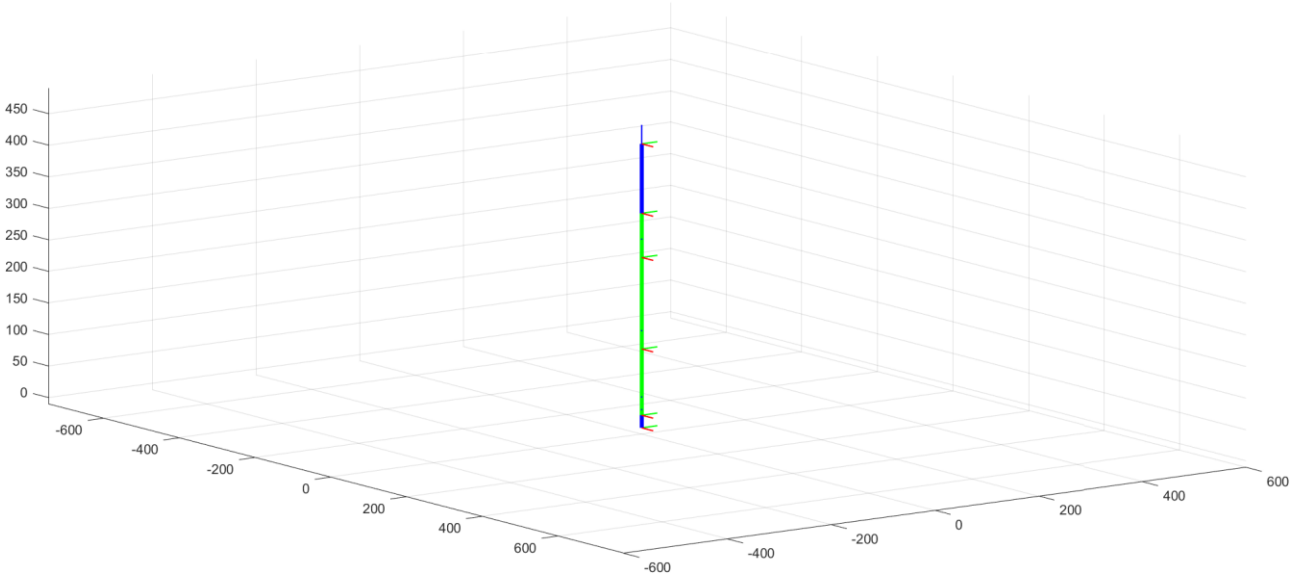


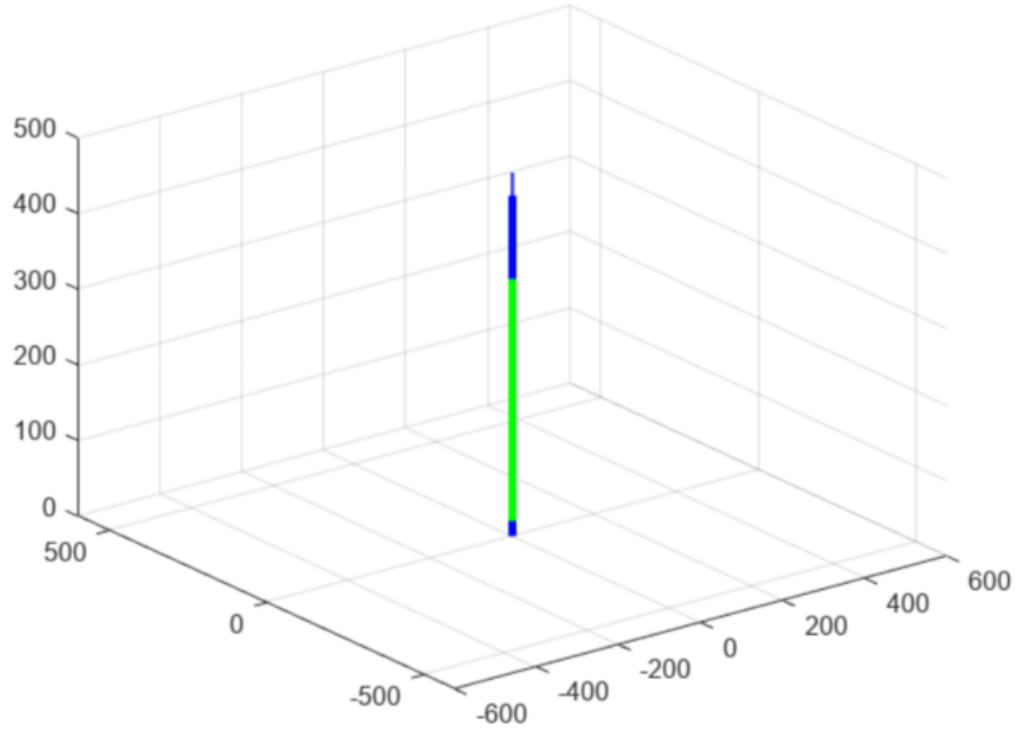
Figura 3.5.8 Reprezentarea configurației (toate cuplele se află în poziția 0)

3.5.3 Pseudo-animație

Animația brațului robotic este realizată printr-o structură de tip buclă for, unde fiecare etapă simulează mișcarea unei singure cuple, în timp ce celelalte sunt menținute constante. Astfel, în prima etapă, se variază doar prima cuplă (articulație), iar celelalte patru sunt fixate la zero. În etapa următoare, se păstrează poziția finală a primei cuple, se variază a doua, iar celelalte (cupla 3, 4 și 5) rămân pe zero. Acest proces se repetă pentru fiecare dintre cele 5 cuple, în ordine, simulând mișcarea treptată a fiecărei articulații în mod individual. Acest mod de abordare oferă o vizualizare clară și secvențială a influenței fiecărei cuple asupra configurației brațului robotic, făcând mai ușoară înțelegerea cinematicii directe.

La fiecare pas, se aplică din nou geometria directă, folosind matricele de transformare pentru a calcula poziția și orientarea fiecărui element al brațului. Acest mod de abordare oferă o

vizualizare clară și secvențială a influenței fiecărei cuple asupra configurației brațului robotic, făcând mai ușor efectului fiecărei mișcări asupra întregului mecanism.



3.5.9 Animație (robotul pleacă de la poziția 0 in toate cuplele și trebuie să se aplice pi/2 pe cuplele q_3 $și$ q_4)

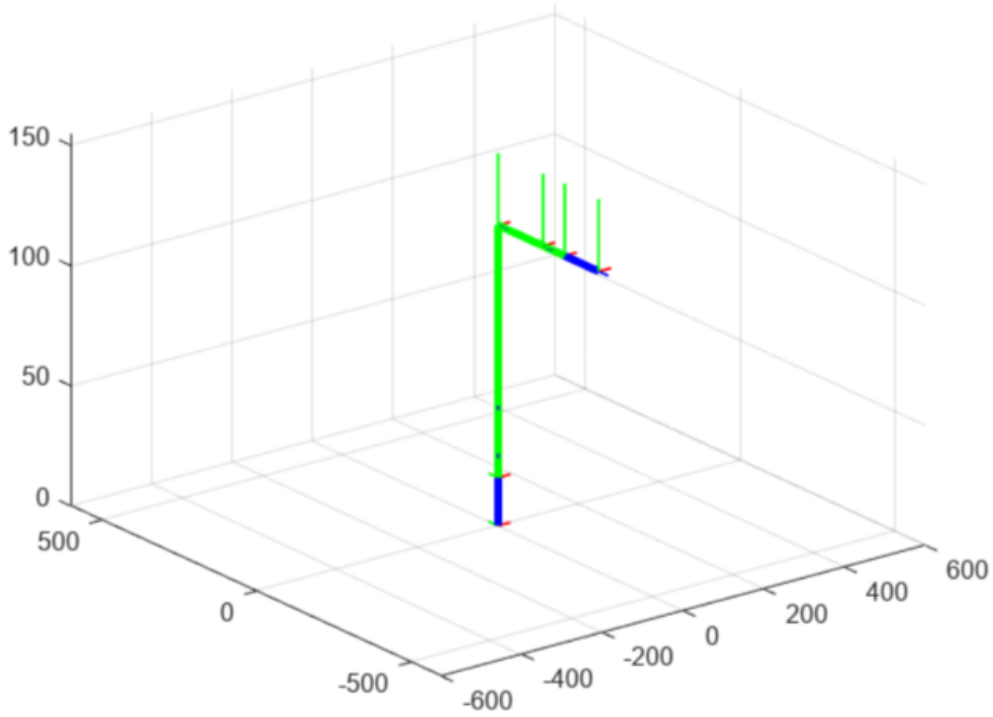


Figura 3.5.10 Animație (q_3 a ajuns la pi/2, iar acum se modifică q_4)

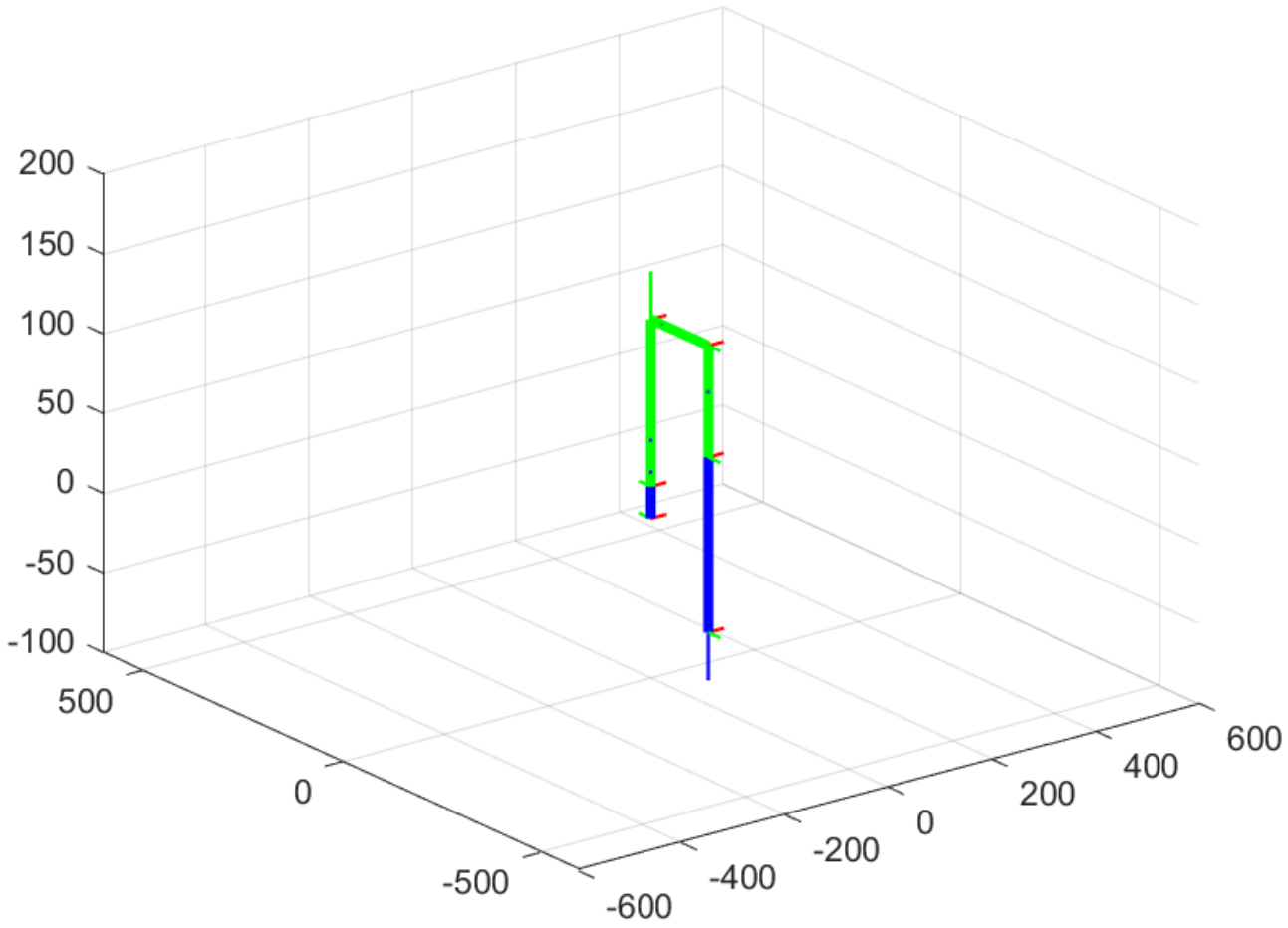


Figura 3.5.11 Animație (q_4 a ajuns la pi/2 și robotul s-a mișcat conform impunerii)

3.5.4 Calculul Jacobianului

Jacobianul reprezintă un concept fundamental în cadrul cinematicii manipulatorilor. Matricea Jacobian poate fi descompusă în două submatrici distincte J_v,J_ω fiecare având dimensiunea 3×n. Submatricea J_v este utilizată pentru determinarea vitezei liniare a efectorului final, în timp ce submatricea J_ω este responsabilă pentru calculul vitezei unghiulare.

Pentru determinarea elementelor matricei Jacobiene J_{6xn}, există mai multe metode, dintre care se va evidenția metoda bazată pe derivarea vectorului de poziție și pe utilizarea versorilor axelor de rotație. Această metodă are ca argumente transformările omogene T_i^o, care descriu poziția și orientarea reperului {i} față de reperul de bază {0}, precum și vectorul q.

$$\begin{Bmatrix} q \\ {}^0_iT \end{Bmatrix} \rightarrow J(q) = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix} \tag{3.0}$$

Date inițiale:

$$\{q, T_i^0, C\} \rightarrow p = T_e^0(1{:}3,4),\ w_i^0 = T_i^0(1{:}3,C_i) \rightarrow J_v(q) = \left(\frac{\partial p}{\partial q_1} \quad \cdots \quad \frac{\partial p}{\partial q_n}\right),\ J_w(q) = [\widehat{w}_1^0 \quad \cdots \quad \widehat{w}_n^0]$$

Unde:

-p este vectorul de poziție al punctului în care se calculează Jacobianul

-$\widehat{w}_i^0$este versorul cuplei C_i pe direcția căruia se realizează mișcarea

Rezultat: J(q) = $\begin{bmatrix} J_v(q) \\ J_w(q) \end{bmatrix}$

Determinarea vectorului p și versorului se realizează utilizând metoda geometriei directe, prezentată în capitolul anterior:

$$ {}^0_P T = \begin{bmatrix} {}^0_P R & p \\ 0 & 1 \end{bmatrix} \rightarrow \text{p} = {}^0_p T(1{:}3,4) \tag{3.1} $$

$$ {}^0_i T = \begin{bmatrix} \hat{x}_i^0 & \hat{y}_i^0 & \hat{z}_i^0 & t_i^0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \rightarrow w_i^0 = \begin{cases} \hat{x}_i^0 \; C_i = R_x \\ \hat{y}_i^0 \; C_i = R_y \\ \hat{z}_i^0 \; C_i = R_Z \\ 0 \; C_i = D_{x,y,z} \end{cases} \tag{3.2} $$

Unde:

-C_i este simbolul cuplei cu numărul i;

$$ \text{J(q)} = \begin{bmatrix} J_v(q) \\ J_w(q) \end{bmatrix} = \begin{bmatrix} \frac{\partial p_e}{\partial q_1} & \frac{\partial p_e}{\partial q_2} & \frac{\partial p_e}{\partial q_3} & \frac{\partial p_e}{\partial q_4} & \frac{\partial p_e}{\partial q_5} \\ w_1^0 & w_2^0 & w_3^0 & w_4^0 & w_5^0 \end{bmatrix} \tag{3.3} $$

$$ {}^0_1 T = \begin{bmatrix} cos(q_1) & -sin(q_1) & 0 & 0 \\ sin(q_1) & cos(q_1) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \rightarrow R_z \rightarrow w_1^0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} $$

$$ {}^0_2 T = \begin{bmatrix} cos(q_1) & -cos(q_2) * sin(q_1) & sin(q_1) * sin(q_2) & 0 \\ sin(q_1) & cos(q_1) * cos(q_2) & -cos(q_1) * sin(q_2) & 0 \\ 0 & sin(q_2) & cos(q_2) & l1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \rightarrow R_x \rightarrow w_2^0 = \begin{bmatrix} cos(q_1) \\ sin(q_1) \\ 0 \end{bmatrix} $$

Și se continuă la fel până la w_5^0, iar pentru p_e se extrag primele 3 rânduri de pe ultima coloană a matricei T_e^0.

$$ \frac{\partial p_e}{\partial q_1} = \begin{bmatrix} \frac{\partial p_x}{\partial q_1} \\ \frac{\partial p_y}{\partial q_1} \\ \frac{\partial p_z}{\partial q_1} \end{bmatrix} $$

Și se continua la fel până la $\frac{\partial p_e}{\partial q_5}$, iar la final se formează Jacobianul de forma:

$$ \text{J(q)(6x5)} = \begin{bmatrix} J_v(q) \\ J_w(q) \end{bmatrix} = \begin{bmatrix} \frac{\partial p_e}{\partial q_1} & \frac{\partial p_e}{\partial q_2} & \frac{\partial p_e}{\partial q_3} & \frac{\partial p_e}{\partial q_4} & \frac{\partial p_e}{\partial q_5} \\ w_1^0 & w_2^0 & w_3^0 & w_4^0 & w_5^0 \end{bmatrix} $$

Matricea Jacobian exprimată simbolic:

$$J(q) = \begin{bmatrix} J_{11} & J_{12} & J_{13} & J_{14} & J_{15} \\ J_{21} & J_{22} & J_{23} & J_{24} & J_{25} \\ J_{31} & J_{32} & J_{33} & J_{34} & J_{35} \\ J_{41} & J_{42} & J_{43} & J_{44} & J_{45} \\ J_{51} & J_{52} & J_{53} & J_{54} & J_{55} \\ J_{61} & J_{62} & J_{63} & J_{64} & J_{65} \end{bmatrix}$$

$$J11 = \cos(q1) \cdot (l2 \cdot \sin(q2) + l3 \cdot \sin(q2 + q3) + l4 \cdot \sin(q2 + q3 + q4) + l5 \cdot \sin(q2 + q3 + q4)$$

$$J12 = \sin(q1) \cdot (l2 \cdot \cos(q2) + l3 \cdot \cos(q2 + q3) + l4 \cdot \cos(q2 + q3 + q4) + l5 \cdot \cos(q2 + q3 + q4)$$

$$J13 = \sin(q1) \cdot (l3 \cdot \cos(q2 + q3) + l4 \cdot \cos(q2 + q3 + q4) + l5 \cdot \cos(q2 + q3 + q4))$$

$$J14 = \sin(q1) \cdot \cos(q2 + q3 + q4) \cdot (l4 + l5)$$

$$J15 = 0$$

$$J21 = \sin(q1) \cdot (l2 \cdot \sin(q2) + l3 \cdot \sin(q2 + q3) + l4 \cdot \sin(q2 + q3 + q4) + l5 \cdot \sin(q2 + q3 + q4)$$

$$J22 = -\cos(q1) \cdot (l2 \cdot \cos(q2) + l3 \cdot \cos(q2 + q3) + l4 \cdot \cos(q2 + q3 + q4) + l5 \cdot \cos(q2 + q3 + q4)$$

$$J23 = -\cos(q1) \cdot (l3 \cdot \cos(q2 + q3) + l4 \cdot \cos(q2 + q3 + q4) + l5 \cdot \cos(q2 + q3 + q4))$$

$$J24 = -\cos(q1) \cdot \cos(q2 + q3 + q4) \cdot (l4 + l5)$$

$$J25 = 0$$

$$J31 = 0$$

$$J32 = -l2 \cdot \sin(q2) - l3 \cdot \sin(q2 + q3) - l4 \cdot \sin(q2 + q3 + q4) - l5 \cdot \sin(q2 + q3 + q4)$$

$$J33 = -l3 \cdot \sin(q2 + q3) - l4 \cdot \sin(q2 + q3 + q4) - l5 \cdot \sin(q2 + q3 + q4)$$

$$J34 = -\sin(q2 + q3 + q4) \cdot (l4 + l5)$$

$$J35 = 0$$

$$J41 = 0$$

$$J42 = \cos(q1)$$

$$J43 = \cos(q1)$$

$$J44 = \cos(q1)$$

$$J45 = \sin(q2 + q3 + q4) \cdot \sin(q1)$$

$$J51 = 0$$

$$J52 = \sin(q1)$$

$$J53 = \sin(q1)$$

$$J54 = \sin(q1)$$

$$J55 = -\sin(q2 + q3 + q4) \cdot \cos(q1)$$

$$J61 = 1$$

$$J62 = 0$$

$$J63 = 0$$

$$J64 = 0$$

$$J65 = \cos(q2 + q3 + q4)$$

3.5.5 Metoda de geometria inversă

Metoda geometriei inverse reprezintă o funcție prin care se determină variabilele generalizate din spațiul articular q, în condițiile în care postura efectorului final este cunoscută sau impusă. Mai exact, aceasta răspunde la întrebarea: care sunt valorile variabilelor generalizate care conduc la o anumită configurație dorită a efectorului final?

Determinarea variabilelor articulare generalizate asociate lanțului cinematic de poziționare:

- implică, de regulă, o soluție numerică, care utilizează concepte asociate matricei Jacobiene, prezentate în capitolul anterior.

Este important de menționat faptul că există anumite posturi ale efectorului final care nu pot fi realizate, indiferent de valorile variabilelor generalizate. Aceste situații sunt strâns legate de conceptul de volum de lucru, care va fi detaliat în capitolele următore.

Se folosește același set de date inițiale ca la primul capitol la care se adaugă:

α - coeficientul de învățare,

ε – eroarea admisibila,

N – numărul de iterații.

Se calculează Jacobianul manipulatorului din capitolul anterior.

Se calculează cu ajutorul metodei geometrice directe postura efectorului: p(k) și eroarea față de poziția dorită a efectorului:

$$\Delta p_k = p_e - \text{p(k)} \tag{4.0}$$

Se calculează, iterativ, valorile variabilelor generalizate ale cuplelor manipulatorului:

$$q_{k+1} = q_k + \alpha * J^{-1}(q_k) * \Delta p_k \tag{4.1}$$

Se calculează, cu ajutorul metodei geometrice directe postura efectorului p(k+1).

Dacă condiția de exacitate, eroarea aproximării numerice, sau cea a maximului de iterații este îndeplinită iterațiile se opresc, altfel se reia ciclul de calcul de la punctul 1:

$$\begin{cases} \Delta p(k+1) = p_e - p(k+1) \leq \varepsilon \\ sau \\ k+1 \geq N \end{cases} \tag{4.2}$$

În final, se obține vectorul variabilelor generalizate q_n.

Se impune P_e = [100,100,100]

α – 0.05

ε - 10^{-3}

n - 100

P = $P_e - p_5$ (p_5 - poziția finală a efectorului)

Condiția de exacititate:

norm(P) < epsilon

J_pos = J(1:3, :) – se aleg primele 3 rânduri din Jacobian doar pentru poziție, nu și pentru orientare.

Calculul iterativ:

Q = Q + alpha * (J_pos' * inv(J_pos * J_pos')) * P;

Inversarea matricei Jacobiene reprezintă un aspect esențial în cadrul cinematicii manipulatorilor, fiind un element central în numeroase metode de analiză și control. Inversarea directă este posibilă în condițiile în care Jacobiana este o matrice pătratică, iar rangul său este maxim, adică egal cu dimensiunea acesteia. În situațiile în care aceste condiții nu sunt îndeplinite – de exemplu, atunci când Jacobiana este dreptunghiulară sau are un rang inferior – se recurge la metode aproximative, cea mai frecvent utilizată fiind pseudo-inversa. Aceasta permite obținerea unei soluții de compromis în cazul sistemelor supradeterminate sau subdeterminate.

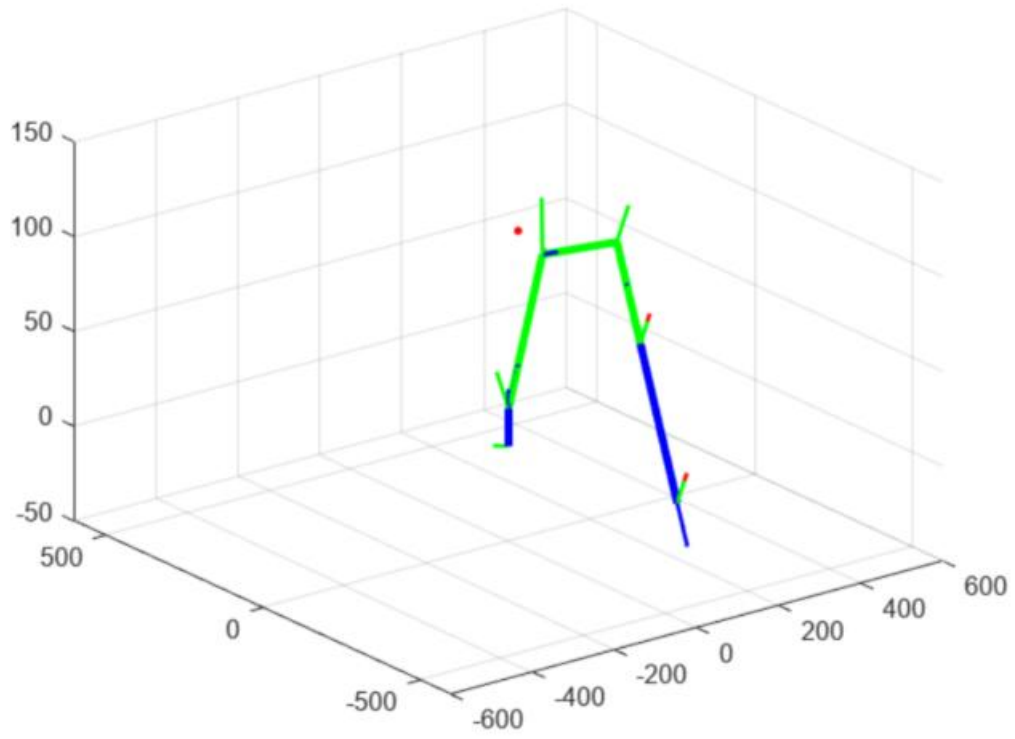


Figura 3.5.12 Geometrie inversă (robotul pleacă de la toate cuplele fiind la pi/4 și trebuie să ajungă în P_e, adică punctul roșu)

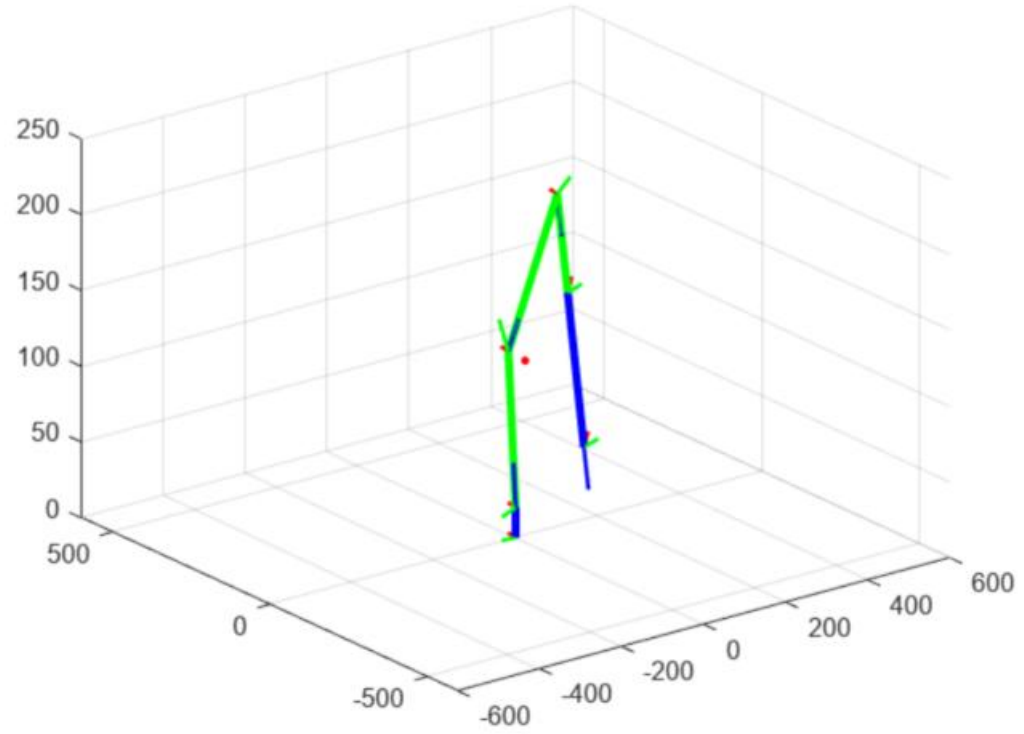


Figura 3.5.13 Geometrie inversă (mișcările robotului la jumătate din iterații)

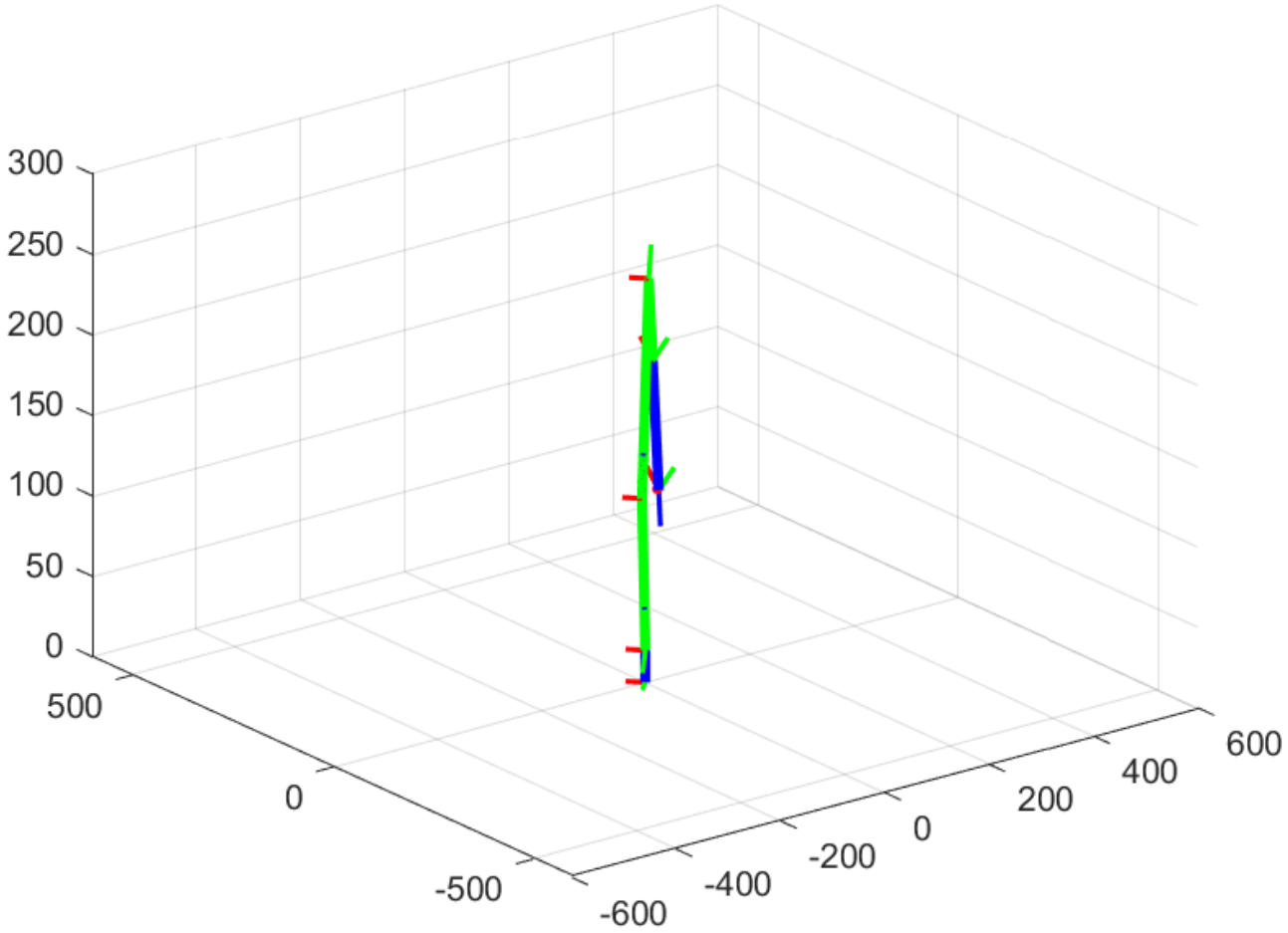


Figura 3.5.14 Geometrie inversă (robotul a ajuns în P_e)

Și vectorul Q = [2.3420, -0.3336, 0.8417, 2.0154, 0.7854]

3.5.6 Pick and Place

Sarcina de tip pick and place constă într-o succesiune de operații standardizate: deplasarea brațului robotic către o poziție inițială pentru preluarea obiectului, transportul acestuia și plasarea lui într-o poziție finală. Din punct de vedere al metodelor de generare a traiectoriilor, se pot evidenția următoarele aspecte:

Această sarcină poate fi interpretată ca o deplasare a efectorului final de la o poziție inițială (corespunzătoare preluării obiectului) către o poziție finală (corespunzătoare plasării obiectului).

Prin aplicarea metodei de cinematică inversă, poziția efectorului final este tradusă în variabilele generalizate ale articulațiilor. Astfel, generarea traiectoriei implică determinarea funcțiilor de mișcare pentru fiecare articulație în parte.

În mod obișnuit, soluția constă în interpolarea valorilor fiecărei variabile articulare între poziția inițială și cea finală.

Traiectoria efectorului între cele două poziții nu este, în general, impusă. Dacă se dorește controlul explicit al traiectoriei în spațiul cartezian, se introduc poziții intermediare, definind astfel un set de traiectorii elementare între posturi succesive.

Condițiile care trebuie respectate în generarea traiectoriei includ pozițiile inițială și finală, durata totală a mișcării, parametrii cinematici (precum vitezele inițiale și finale, de obicei nule), viteza maximă admisă și, în unele cazuri, accelerarea minimă.

Această metodă permite calculul traiectoriilor de referință necesare pentru ca sistemul robotic să execute corect sarcina.

Dintre metodele existente, în această lucrare va fi prezentată soluția bazată pe utilizarea funcțiilor polinomiale.

Interpolarea prin funcții polinomiale este una dintre cele mai frecvent utilizate tehnici pentru generarea traiectoriilor în spațiul articular al roboților industriali, având ca scop determinarea unei funcții de formă:

$$q(t) = a_3 * t^3 + a_2 * t^2 + a_1 * t + a_0 \quad (5.0)$$

unde q(t) reprezintă poziția unei variabile generalizate în funcție de timpul t, iar coeficienții $a_0, a_1, a_2, a_3,$ sunt calculați astfel încât să îndeplinească condiții specifice de continuitate și performanță cinematică.

$$\begin{cases} a_3 = -2\dfrac{q_\Delta - q_0}{\Delta^3} \\ a2 = 3\dfrac{q_\Delta - q_0}{\Delta^2} \\ a1 = 0 \\ a0 = q_0 \end{cases} \quad (5.1)$$

Unde:

-q_Δ - poziția finală

-q_0 – poziția inițială

Pentru o deplasare a robotului de la punctul P_e= [100,100,100] la [150,150,150] aveam următoarele:

q_0=[2.3420; -0.3336; 0.8417; 2.0154; 0.7854] (unghiurile la fiecare cuplă in parte in radiani)

q_Δ=[2.3469; -0.1764; 0.7407; 1.5755; 0.7854]

Cu următoarele figuri pentru poziție, viteză și accelerație:

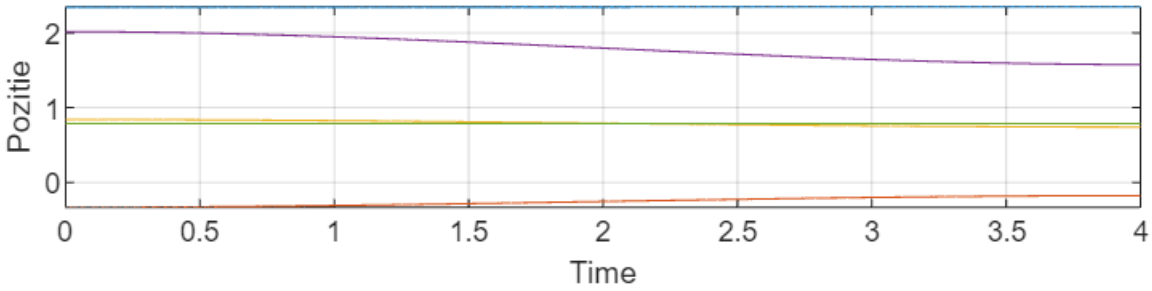


Figura 3.5.15 Poziția Pick and Place

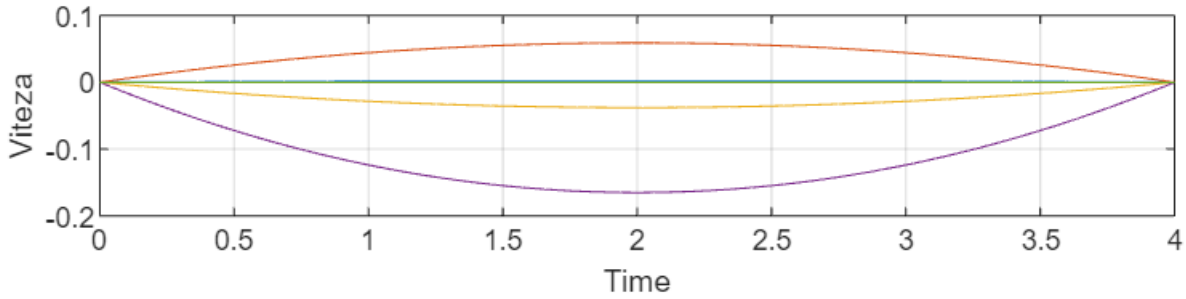


Figura 3.5.16 Viteza Pick and Place

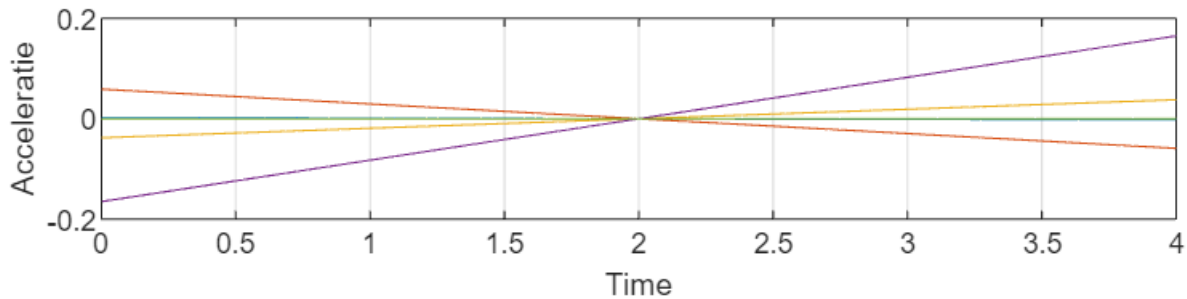


Figura 3.5.17 Accelerația Pick and Place

3.5.7 Cinematică directă

Metoda se reduce la aplicarea relației cinematice directe a vitezelor.

$$\begin{bmatrix} v_p \\ \omega_p \end{bmatrix} = J(q) * \dot{q} \tag{6.0}$$

Astfel, dacă sunt cunoscute traiectoriile variabilelor generalizate în spațiul articular (de exemplu, traiectorii pentru operațiuni de tip pick and place), se pot calcula vitezele punctelor cheie ale structurii manipulatorului, inclusiv ale efectorului final.

Jv = J(1:3, :) – primele 3 linii din Jacobian pentru viteza liniară

Jw = J(4:6, :) – ultimele 3 linii din Jacobian pentru viteza unghiulară

Vitezele, adică $\dot{q}$, si pozițiile sunt extrase din traiectoria Pick and Place.

La fiecare pas se calculează Jacobianul si se aplică (relația).

viteza liniara = [viteza liniara, Jv * viteza(:, i)];

omega = [omega, Jw * viteza(:, i)];

După ce bucla s-a încheiat:

- Viteza liniara va fi o matrice 3×N— vectorii vitezei liniare în timp a efectorului.
- omega va fi o matrice 3×N— vectorii vitezei unghiulare în timp a efectorului.

```
viteza_lineara = 3×100
         0    0.8424    1.6674    2.4745    3.2637    4.0346    4.7869    5.5206    6.2354    6.9311 ...
         0    0.8333    1.6492    2.4477    3.2285    3.9912    4.7359    5.4622    6.1700    6.8592
         0    0.6220    1.2317    1.8298    2.4166    2.9924    3.5578    4.1130    4.6583    5.1939

omega = 3×100
         0    0.0040    0.0079    0.0118    0.0156    0.0192    0.0228    0.0264    0.0298    0.0332 ...
         0   -0.0041   -0.0082   -0.0121   -0.0160   -0.0198   -0.0235   -0.0271   -0.0307   -0.0341
         0    0.0001    0.0001    0.0002    0.0003    0.0004    0.0004    0.0005    0.0005    0.0006
```

Figura 3.5.18 Cinematica directă

3.5.8 Cinematică inversă

Ecuația cinematică inversă a vitezei:

$$\dot{q} = J^{+}(q) * v \tag{7.0}$$

Unde:

-$\dot{q}$ – viteza generalizată

-J^{+}(q) – pseudo-inversa matricei Jacobiene

-v = $\begin{bmatrix} v_p \\ \omega_p \end{bmatrix}$– viteza spatială a capătului efector formată din viteza liniară si viteza unghiulară (din cinematica directă)

viteza_generalizata = 5×100

0	0.0001	0.0001	0.0002	0.0003	0.0004	0.0004	0.0005	0.0005	0.0006 ...
0	0.0024	0.0047	0.0069	0.0091	0.0113	0.0134	0.0155	0.0175	0.0195
0	-0.0015	-0.0030	-0.0045	-0.0059	-0.0073	-0.0086	-0.0100	-0.0113	-0.0125
0	-0.0066	-0.0131	-0.0194	-0.0256	-0.0316	-0.0376	-0.0434	-0.0490	-0.0545
0	-0.0000	-0.0000	0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	0.0000

Figura 3.5.19 Viteza generalizată din cinematică inversă

viteza = 5×100

0	0.0001	0.0001	0.0002	0.0003	0.0004	0.0004	0.0005	0.0005	0.0006 ...
0	0.0024	0.0047	0.0069	0.0091	0.0113	0.0134	0.0155	0.0175	0.0195
0	-0.0015	-0.0030	-0.0045	-0.0059	-0.0073	-0.0086	-0.0100	-0.0113	-0.0125
0	-0.0066	-0.0131	-0.0194	-0.0256	-0.0316	-0.0376	-0.0434	-0.0490	-0.0545
0	0	0	0	0	0	0	0	0	0

Figura 3.5.20 Viteza din traiectoria de tip Pick and Place

3.5.9 Volumul de lucru

Volumul de lucru reprezintă totalitatea pozițiilor în spațiu tridimensional pe care le poate atinge capătul efector (end-effector-ul) al robotului, în funcție de variația unghiurilor articulațiilor. În acest scop, se utilizează o metodă discretă de eșantionare a spațiului articulațiilor, în care unghiurile articulațiilor sunt variate în mod sistematic, iar pozițiile corespunzătoare în spațiu sunt calculate prin aplicarea geometriei directe.

Inițial, se definește o rezoluție de eșantionare prin variabila n, care determină câte valori sunt luate în considerare pentru fiecare unghi. Articulația de la bază (q_1) este rotită într-un interval de 180° (de la -90° la +90°), descriind astfel un semicerc complet în planul orizontal. Celelalte articulații (q2, q3 și q4) sunt variate în intervalul [-90°, 0°], limitând astfel mișcarea brațului doar în partea frontală a robotului. Unghiul q5, asociat prehensorului, este menținut constant (0 radiani), deoarece acest element nu este luat în calcul în analiza volumului de lucru.

Pentru fiecare combinație de unghiuri generate, se apelează geometria directa, care returnează poziția capătului efector în sistemul de coordonate global, în urma aplicării transformărilor succesive. Din vectorul rezultat, sunt extrase coordonatele poziției (X, Y, Z), iar punctele situate în spatele bazei robotului (având componenta Y negativă) sunt eliminate, pentru a limita analiza doar la partea frontală.

Toate pozițiile valide sunt stocate într-o matrice, iar la final sunt reprezentate grafic în spațiul tridimensional. Această reprezentare oferă o imagine intuitivă asupra capacității robotului de a ajunge în diverse puncte din spațiu, fiind extrem de utilă în etapa de proiectare, planificare a mișcărilor sau optimizare a configurației robotului pentru diverse aplicații.

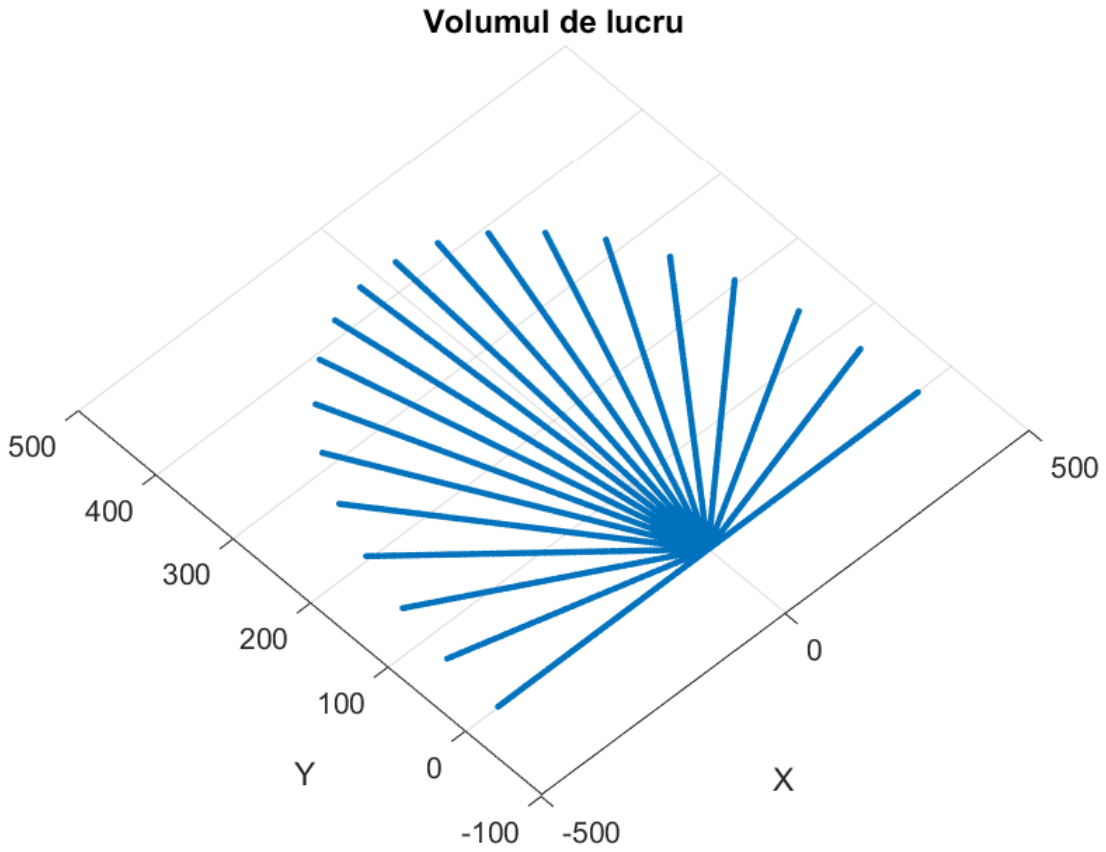


Figura 3.5.21 Volumul de lucru (vedere de sus)

3.6 COMPONENTE FOLOSITE

Pentru toate articulațiile brațului s-a optat pentru servo-motorul MG996R, ale cărui caracteristici sunt prezentate în Tabelul 3.6.1.



Figura 3.6.1 Servomotor MG966R [15]

Principiul de funcționare a unui astfel de servo-motor este prezentat in capitolul 2.4.1 Principiul de funcționare al unui servo-motor.

Tabel 3.6.1 Caracteristici servomotor MG966R [15]

Tensiune de operare	5V
Curent	2.5A (la 6V)
Cuplu de blocare	9.4 kg/cm (la 4.8V)
Cuplu maxim de blocare	11 kg/cm(la 6V)
Rotație	$0^o - 180^o$
Greutatea motorului	55g
Viteza de operare	0.17sec/60°

Șasiul folosește motoare DC cu reductor controlate prin driverul L9110 cu următoarele specificații:

Tabel 3.6.2 Caracteristici motor DC și driver L9110 [16]

Tensiune de alimentare motor	3V-8V
Curent de funcționare	250mA per motor (la 7.2V)
Cuplu maxim	1.5 kg*cm
Viteză fără sarcină	250rpm
Tensiune de alimentare	3V-12V

Pentru alimentarea întregului sistem, care include servo-motoarele MG996R, motoarele de pe șasiu, un Raspberry Pi 5 și camera video atașată acestuia, s-a optat pentru utilizarea unei surse de alimentare de 5V cu un curent maxim de 20A. Această alegere este justificată prin estimarea consumului total:

- 5 servo-motoare MG996R × max. 2.5A = 12.5A (în sarcină maximă)
- 2 motoare DC pentru șasiu × ~0.35A = ~0.7A
- Raspberry Pi 5 cu cameră = ~4A (în funcționare intensă)

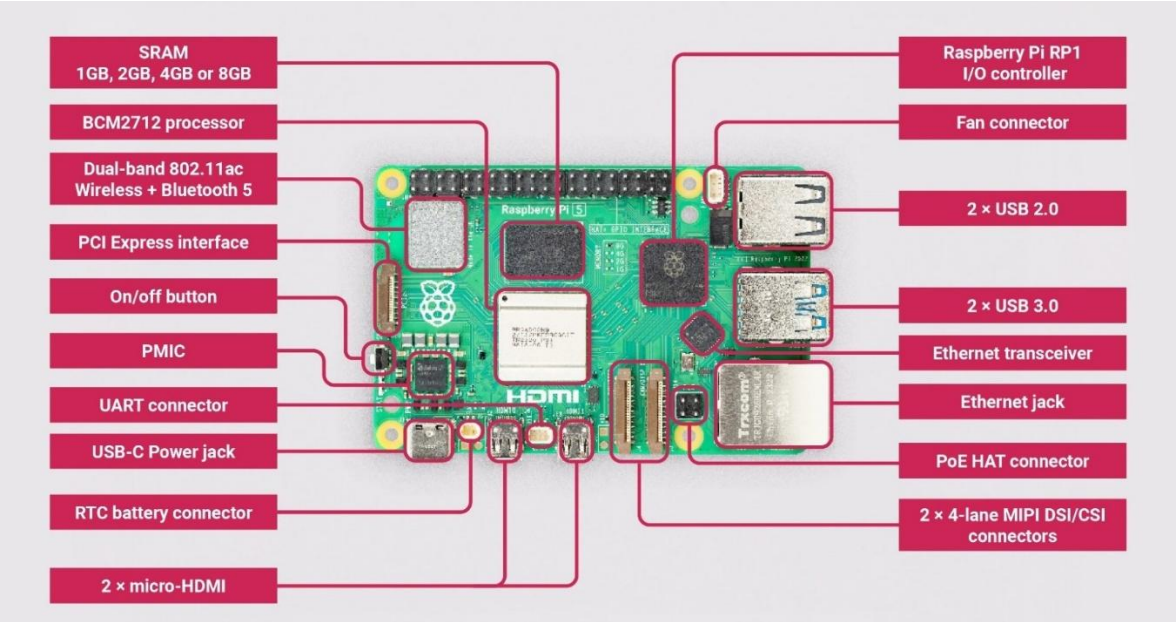


Figura 3.6.2 Raspberry Pi 5 [19]

Consum total estimat:

$$12.5A + 0.7A + 4A = \sim 17.2A$$

Prin urmare, sursa de 5V / 20A oferă o marjă de siguranță suficientă pentru funcționarea stabilă a întregului sistem, chiar și în condiții de solicitare maximă.

Este important de menționat că placa de dezvoltare PCA9685, utilizată pentru controlul servo-motoarelor, nu a fost inclusă în calculul consumului total de curent. Aceasta are un consum propriu redus, de ordinul câtorva zeci de miliamperi, neinfluențând semnificativ dimensionarea sursei de alimentare. Rolul principal al plăcii este de a genera semnalele PWM necesare pentru controlul motoarelor, fără a furniza direct curentul de acționare, care este preluat din sursa de 5V/20A. Astfel, consumul acesteia este considerat neglijabil în raport cu cel al componentelor de putere ale sistemului.

Cablajul utilizat pentru alimentarea sistemului trebuie să fie dimensionat corespunzător pentru curentul maxim estimat de aproximativ 17.2A, pentru a preveni căderile de tensiune și încălzirea excesivă a cablurilor. Pentru un curent de această mărime, se recomandă utilizarea unui cablu cu secțiune transversală de minimum 1.5 mm², de preferat 2.5 mm², din cupru, cu izolație adecvată pentru 5V și condițiile de mediu ale aplicației. Acest lucru asigură o rezistență redusă și o disipare minimă a căldurii, garantând astfel siguranța și stabilitatea alimentării întregului sistem. De asemenea, este important ca traseul cablului să fie cât mai scurt și bine protejat mecanic.



Figura 3.6.3 Sursă 5V 20A [20]

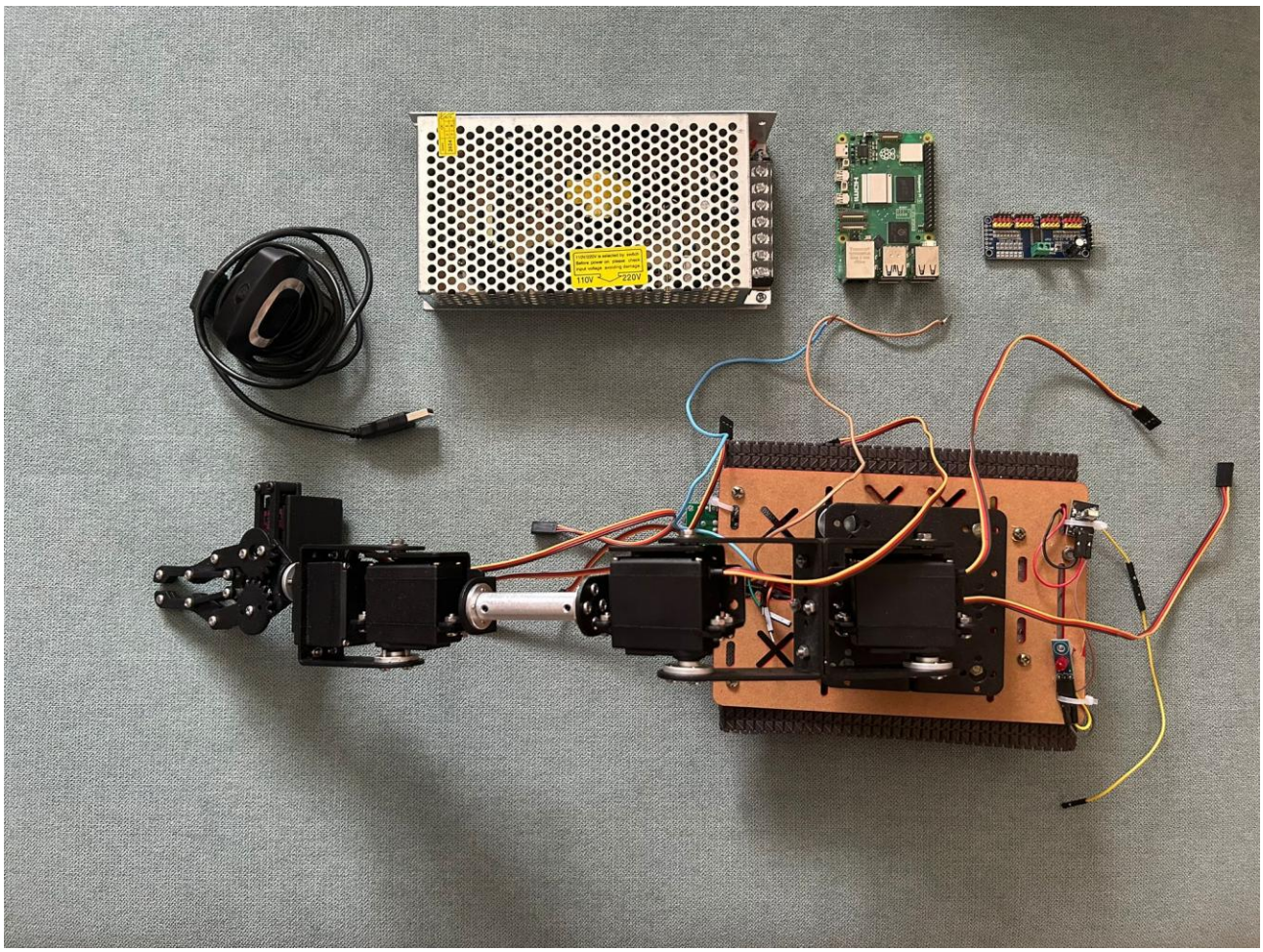


Figura 3.6.4 Vedere de ansamblu asupra componentelor sistemului

4 REZULTATE EXPERIMENTALE

În acest capitol sunt prezentate rezultatele obținute în urma testării sistemului de recunoaștere vizuală și a platformei de orientare robotică. Sunt analizate comportamentele observate atât în detecția deșeurilor cu ajutorul modelului YOLO, cât și în identificarea markerilor ArUco pentru calibrarea camerei și estimarea poziției obiectelor. De asemenea, sunt evidențiate atât performanțele pozitive, cât și limitările sistemului în scenarii generale sau speciale.

4.1 TESTAREA DETECTIEI CU YOLOV8N

4.1.1 Cadrul experimental

Sistemul de detecție a fost testat utilizând o cameră montată pe o platformă robotică. Camera este conectată la un microcomputer (Raspberry Pi), care rulează în timp real modelul YOLOv8 antrenat pe un set de date ce conține imagini cu diferite tipuri de deșeuri: PET-uri, doze de aluminiu, sticle, carton.

Testele au fost realizate într-un mediu controlat, dar cu variabilitate naturală – diferite tipuri de iluminare, fundaluri, poziții ale obiectelor și grad de deformare a acestora.

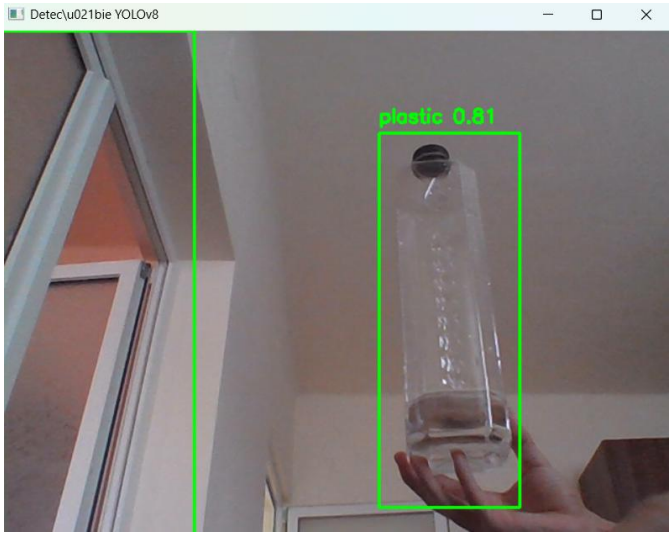


Figura 4.1.1 Detecție PET

4.1.2 Evaluarea rezultatelor

Modelul a reușit să detecteze majoritatea obiectelor testate în condiții normale de iluminare și poziționare, însă nu fără erori. Sistemul a generat predicții false pozitive, adică a detectat obiecte care nu existau cu adevărat sau care nu aveau legătură cu categoria deșeurilor urmărite. De exemplu, în figura 4.1.1, pe lângă detecția corectă a PET-ului, apare și o detecție suplimentară eronată, pe un obiect care nu corespunde niciunei clase antrenate.

Acest tip de eroare poate fi cauzat de:

- fundaluri complexe sau texturate, care induc rețeaua în eroare;
- forma obiectelor sau umbre care seamănă cu obiectele de interes;
- un set de date insuficient de variat și curat.
- dimensiunea pozelor de antrenare.

Aceste rezultate evidențiază necesitatea de a rafina procesul de antrenare, inclusiv prin curățarea dataset-ului de exemple ambigue sau prin augmentarea acestuia cu mai multe imagini din medii reale, în care apar astfel de ambiguități.

Printre cele mai relevante cazuri speciale observate se numără:

1. Obiecte deformate sau rupte:

2. PET-uri zdrobite, doze turtite, sticle sparte și carton mototolit nu mai păstrează forma obișnuită, ceea ce face dificilă recunoașterea de către model.

Sistemul a eșuat frecvent în identificarea acestor obiecte, întrucât setul de date de antrenament nu a conținut suficiente exemple de acest tip.

4.2 TESTAREA MARKERILOR ARUCO PENTRU CALIBRARE ȘI POZIȚIONARE

4.1.1 Cadrul experimental

Pentru testarea funcționalității markerilor ArUco în cadrul sistemului, s-a realizat un experiment simplu de validare a detectării acestora și a estimării poziției și orientării în spațiu.

S-au testat două scenarii distincte:

- **Scenariul 1** – Camera **calibrată**, folosind un model de calibrare obținut cu ajutorul unui grid tip tablă de șah și algoritmii OpenCV.
- **Scenariul 2** – Camera **necalibrată**, folosind setările implicite, fără corectarea distorsiunilor de lentilă.

În cazul camerei calibrate, este vizibilă și estimarea orientării markerului în spațiu, reprezentată grafic prin axe colorate (X, Y, Z). În cazul camerei necalibrate, deși markerul este detectat, orientarea nu este estimată corect sau nu este afișată deloc.

Scopul acestui test este de a demonstra influența procesului de calibrare asupra preciziei sistemului de viziune, în special în aplicații care implică poziționare și orientare în spațiu 3D.

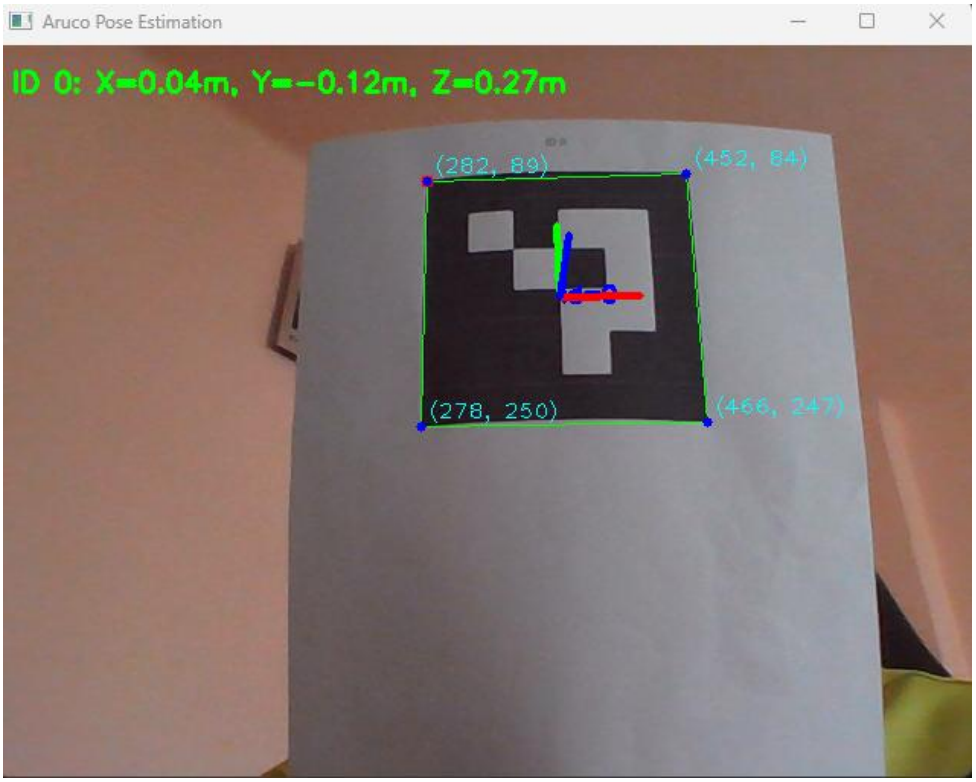


Figura 4.2.1 Cameră calibrată

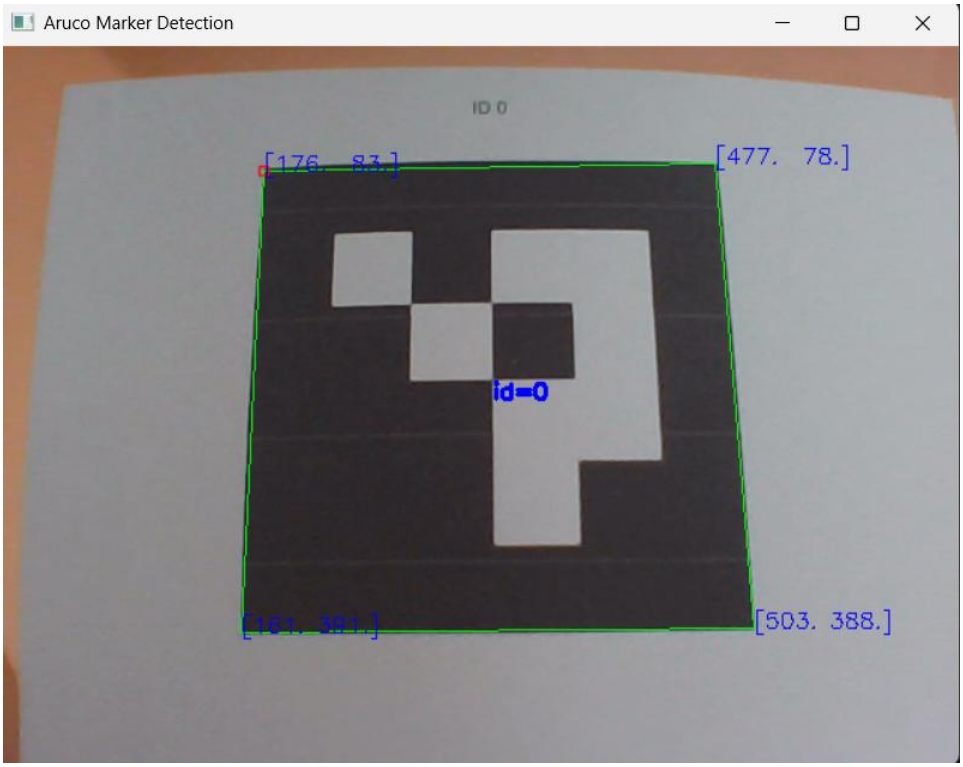


Figura 4.2.2 Cameră necalibrată

5 CONCLUZII

5.1 CONCLUZII GENERALE

Toate metodele și algoritmii prezentați în această lucrare au fost utilizați cu scopul de a controla poziția și orientarea brațului robotic, în raport cu obiectele detectate în imagine. Interacțiunea dintre componentele de viziune și cele mecanice a permis realizarea unui sistem complet de detecție și manipulare.

O etapă esențială în buna funcționare a sistemului a fost calibrarea camerei, care a permis corectarea distorsiunilor optice și obținerea unei relații corecte între coordonatele din imagine (pixeli) și poziția reală a obiectelor în spațiul tridimensional. Pe baza acestor parametri optici (matricea camerei și coeficienții de distorsiune), sistemul a putut interpreta corect poziția obiectelor detectate cu ajutorul markerilor ArUco.

Astfel, prin identificarea markerilor ArUco în imagine, sistemul determină poziția 3D a markerului (punctul de pick) și orientarea acestuia (rotirea piesei în spațiu), folosind funcții dedicate care estimează vectorii de rotație și translație. Această informație este important pentru ca brațul robotic să poziționeze corect efectorul final (gripper-ul) în raport cu piesa de preluat. Precizia acestei localizări este posibilă doar datorită procesului corect de calibrare a camerei.

De asemenea, pentru recunoașterea categoriilor de materiale (plastic, metal, hârtie, aluminiu), s-a utilizat o rețea neuronală YOLOv8, antrenată cu imagini preluate, etichetate manual. Acest proces a avut scopul de a oferi un sistem de sortare, în care fiecare obiect este identificat și clasificat înainte de a fi preluat.

Din punct de vedere geometric si cinematic, brațul robotic a fost modelat folosind transformări omogene. S-a construit și utilizat matricea Jacobiană, care a permis calculul

variațiilor unghiurilor necesare între două poziții cunoscute ale efectorului, asigurând o mișcare lină și coordonată a brațului robotic.

5.2 CONTRIBUȚII PERSONALE

Contribuțiile personale realizate în cadrul lucrării sunt următoarele:

- Modelarea geometriei si cinematicii a brațului, utilizând matricele de transformare omogenă pentru fiecare articulație.
- Calculul și aplicarea Jacobianului pentru determinarea unghiurilor intermediare dintre două poziții ale efectorului final, facilitând mișcări continue și precise.
- Calibrarea camerei folosind un panou tip tablă de șah și algoritmi OpenCV, obținând parametrii interni (cameraMatrix, distorsion coefficients), care au permis corecția imaginilor și estimarea corectă a pozițiilor reale în spațiu.
- Folosirea markerilor ArUco pentru identificarea poziției 3D și a orientării unui obiect în imagine. Pe baza valorilor de rotație și translație estimate pentru fiecare marker, sistemul determină punctul de preluare și înclinarea necesară a brațului robotic.
- Antrenarea unei rețele YOLOv8, de la zero, utilizând un set de imagini etichetate pentru clasele de interes: plastic, aluminiu, metal și hârtie. Integrarea acestui model în sistem a permis declanșarea acțiunilor de sortare în funcție de tipul materialului detectat.
- Implementarea întregului sistem de control pe platforma Raspberry Pi 5, incluzând achiziția video, procesarea imaginilor, detecția markerilor și a obiectelor, precum și controlul motoarelor.

5.3 POSIBILITATI DE DEZVOLTARE ULTERIOARA

Proiectul realizat oferă o bază solidă pentru extinderea și rafinarea funcționalităților sistemului robotic, atât din punct de vedere software, cât și hardware. Mai jos sunt prezentate direcții concrete de dezvoltare viitoare:

Extinderea și diversificarea setului de date utilizat pentru antrenarea rețelei YOLO, prin capturarea unui număr mai mare de imagini, în diferite condiții de iluminare, orientare și fundal. Un set de date mai bogat și mai variat va permite rețelei să generalizeze mai bine și să detecteze corect obiectele chiar și în situații neprevăzute.

Acordarea unei atenții sporite rezoluției imaginilor din setul de antrenament, deoarece dimensiunea și calitatea acestora influențează direct precizia detecției YOLO. Imagini prea mici sau cu detalii insuficiente pot reduce considerabil acuratețea predicțiilor.

Continuarea simulării cinematicii și trecerea la modelarea dinamică a brațului robotic, pentru a include aspecte precum inerția, forțele și cuplurile în controlul mișcării. Acest pas va permite dezvoltarea unor scheme avansate de control adaptiv, potrivite pentru aplicații în medii reale, nestructurate.

Extinderea rolului Jacobianului: în stadiul actual, Jacobianul este utilizat pentru a calcula unghiurile necesare deplasării între două poziții. Pe viitor, acesta poate fi folosit pentru a extrage orientarea end-effectorului, astfel încât sistemul să nu mai depindă de markerii ArUco pentru determinarea orientării obiectului.

Eliminarea completă a markerilor ArUco, prin estimarea poziției și orientării obiectului direct din conturul detectat de YOLO. Prin analiza proporțiilor, dimensiunii și poziției bounding box-ului în imagine, alături de datele calibrării camerei, sistemul poate reconstrui poziția 3D aproximativă a obiectului pentru a executa operațiunea de preluare.

6 BIBLIOGRAFIE

[1] Terex ZenRobotics, *Soluții robotice pentru reciclare.* Home | ZenRobotics

[2] Waste Robotics, *AI Sorting Robot for Heavy Waste Separation*, Waste Robotics. Waste Robotics - Ai Sorting Robot for Heavy Waste Separation

[3] Recycleye, *Soluții robotice pentru sortarea deșeurilor*, Recycleye – Home. Home - Recycleye

[4] Everyday Robots, *Deep RL at Scale: Sorting Waste in Office Buildings with a Fleet of Mobile Manipulators*, Robotics at Google, 2023.

[5] Md. Shahariar Nafiz, Abdullah Al Juabir, Shuvra Smaran Das, Dip Nandi, Md. Kishor Morol, *ConvoWaste: An Automatic Waste Segregation Machine Using Deep Learning*, American International University Bangladesh (AIUB), 2023.

[6] Abin Timilsina, *YOLOv8 Architecture Explained!*, Medium, 17 martie 2024. YOLOv8 Architecture Explained!. What is YOLOv8 ? | by Abin Timilsina | Medium

[7] Ultralytics, *How do I train a YOLOv8 model?*, Ultralytics Documentation, 2024. Explore Ultralytics YOLOv8 - Ultralytics YOLO Docs

[8] OpenCV, *ArUco Marker Detection*, OpenCV Documentation, 2024. OpenCV: Detection of ArUco Markers

[9] Nikhil Venkat Kumsetty, *TrashBox: Dataset of Trash Objects for Waste Classification and Detection*, GitHub, 2023. GitHub - nikhilvenkatkumsetty/TrashBox: Dataset of trash objects for waste classification and detection

[10] CVAT.ai, *Computer Vision Annotation Tool*, 2024. Leading Image & Video Data Annotation Platform | CVAT

[11] Pozna C., *Bazele cinematicii roboților industriali – Modelul geometric*. Robotics

[12] Pozna C., *Bazele cinematicii roboților industriali – Modelul cinematic*. Robotics

[13] Canyon, *720p HD Webcam C3 (CNE-CWC3N)*. Webcam 720p HD Canyon C3 - Canyon

[14] Calib.io, *Camera Calibration Pattern Generator*. Camera Calibration Pattern Generator – calib.io

[15] Components101, *MG996R Servo Motor Datasheet, Wiring Diagram & Features.* MG996R Servo Motor Datasheet, Wiring Diagram & Features

[16] Robofun, *Șasiu tanc robotic 2 motoare*. Sasiu tanc robotic 2 motoare

[17] Nicolai Høirup Nielsen (niconielsen32), GitHub. niconielsen32 (Nicolai Høirup Nielsen) · GitHub

[18] Grigorescu S., *Sisteme de Vedere Artificială, Optică*, Universitatea Transilvania din Brașov. Curs_06_SVA_2016

[19] Hackatronic, *Raspberry Pi 5: Pinout, Specifications & Pricing – A Complete Guide*. Raspberry Pi 5 Pinout, specifications, Pricing A Complete Guide

[20] Vectro (ElectroAZ), *Sursă de alimentare carcasă metalică interior 220V–5V, 100W, 20A.* Sursa Alimentare 5V, Metalică 20A – 100W ElectroAZ - Vectro

REZUMAT

Lucrarea de față prezintă realizarea unui robot mobil prevăzut cu un braț robotic, conceput pentru a îndeplini sarcini de identificare, preluare și sortare a deșeurilor. Sistemul este coordonat de un microcomputer Raspberry Pi, care gestionează atât componentele de recunoaștere vizuală, cât și controlul mișcărilor mecanice.

Pentru identificarea obiectelor, se utilizează o cameră web și un model de detecție bazat pe YOLOv8n, antrenat pentru recunoașterea diferitelor tipuri de deșeuri. Obiectele vizate sunt marcate cu markerele ArUco, din care se extrag poziția și orientarea în spațiu. Aceste informații sunt folosite pentru a calcula mișcările necesare ale brațului robotic în vederea manipulării corecte.

Mișcarea brațului este realizată cu ajutorul unor servomotoare, controlate prin intermediul modulului PCA9685, capabil să genereze semnalele PWM corespunzătoare. Întregul sistem este alimentat de la o sursă de tensiune externă, dimensionată astfel încât să asigure funcționarea stabilă a tuturor componentelor, inclusiv a platformei mobile, brațului robotic și unității de procesare.

Proiectul a fost construit pas cu pas, începând cu alegerea și testarea componentelor hardware, continuând cu dezvoltarea algoritmilor de control, și finalizând cu integrarea și verificarea funcționării într-un mediu controlat. Rezultatele obținute în urma testelor arată că sistemul este capabil să recunoască obiectele, să le preia și să le plaseze în funcție de categoria din care fac parte, conform obiectivelor propuse.

ABSTRACT

This paper presents the development of a mobile robot equipped with a robotic arm, designed to perform tasks involving the identification, pickup, and sorting of waste materials. The system is controlled by a Raspberry Pi microcomputer, which manages both the visual recognition components and the mechanical motion control.

Object identification is performed using a webcam and a detection model based on YOLOv8n, trained to recognize different types of waste. The targeted objects are marked with ArUco markers, from which spatial position and orientation are extracted. This information is then used to compute the necessary movements of the robotic arm for accurate manipulation.

The arm's movement is achieved using servomotors, controlled via the PCA9685 module, which generates the appropriate PWM signals. The entire system is powered by an external power supply, properly dimensioned to ensure stable operation of all components, including the mobile platform, robotic arm, and processing unit.

The project was developed step by step, starting with the selection and testing of hardware components, followed by the development of control algorithms, and finalized with the integration and verification of the system in a controlled environment. The results obtained from testing show that the system can recognize objects, picking them up, and placing them according to their category, in line with the proposed objectives.